The Benchmark Chooses the Winner:

Measuring, Tuning, and Composing Small Safety Guards
in High-Compliance Regulated Domains

Reza Rahimi
JazzX AI, Los Altos, CA, USA
reza.rahimi@jazzx.ai

July 17, 2026

Abstract

A safety *guard* — a small model that labels each incoming request `safe` or `unsafe` before an assistant acts on it — is usually chosen by whichever fine-tune scores best on a public benchmark. That score is *co-produced* by the benchmark, the training objective, the decision threshold, and the domain, so the leaderboard-best guard can be the wrong guard for the traffic you serve. This bites hardest in high-compliance domains — finance, health, law, mortgage lending — where the dangerous request is often *polite, compliant-looking* text (e.g. a mortgage inquiry that quietly steers by a protected class) that a benchmark-tuned guard never learned to catch.

We measure what a fine-tune actually buys by comparing each guard only against its *own* pre-tuning base on identical rows, separating *represented* (trained-on) from *transfer* (held-out) sources. Four findings on one fixed panel of compact checkpoints: (i) fine-tuning *specializes* — it lifts represented ranking to a shared ceiling (+0.3234 macro-AP) but changes transfer by only −0.0589 on average, pooling opposing per-checkpoint effects (+0.04 to −0.15), and, at a calibrated threshold, more than *triples* the pooled false-alarm rate on unseen traffic while dropping hard-attack recall; (ii) a retraining-free base+adapter *composition* recovers much of the lost transfer (+0.076 vs. the fine-tune) at one extra inference pass; (iii) a *preregistered confirmatory* study over ten checkpoints (including six released vendor guards) confirms the same specialization even for purpose-built guards, while KL-regularized SFT preserves transfer but at a real represented cost — a tradeoff dial, not a free win; and (iv) in regulated domains, evaluated base-only and zero-shot, the numerically top-ranked guard *differs* between the mortgage benchmark and the finance/health/law arm (within which one guard leads all three verticals; CIs overlap), and a protected-token probe separates guards that aggregate ranking rates alike (directional; three public-test pairs).

The contribution is a measurement-and-decision *workflow* for choosing guards in regulated products — compare to your own base, gate on both represented and transfer, recalibrate, and treat repairs as gated candidates — not a new winning model. Acts I–III are retrospective on an inspected panel; only the adaptation study is confirmatory; the domain arms are base-only and zero-shot; no causal or fair-lending claim is licensed. Code, data manifests, and the frozen benchmark are public (Section 9; `https://github.com/rrahimi-uci/guard-ranking-fragility`).

Contents

1 Introduction: why "is this guard good?" is the wrong question

In one paragraph

A "guard" is a small model that reads an incoming request and labels it `safe` or `unsafe` before an assistant acts on it. The common recipe is: take a general chat model, fine-tune it into a guard, and report its benchmark score — usually compared against *other* models. That hides the quantity a practitioner actually needs: what did the fine-tune change relative to the *same model before tuning*, and does that change survive on data the guard never trained on? This report answers that at two scales (does tuning transfer? and can we *compose* to recover it without retraining?) and then asks whether any of it holds where it matters most — *regulated domains* where a violation can read as perfectly polite text.

A guard chosen on a leaderboard — or "improved" by a quick fine-tune — can quietly do three expensive things at once in production: **(1)** raise false alarms that block legitimate users (pooled transfer false alarms climb 4.3% → 17.0%); **(2)** miss the hardest attacks it was meant to stop (HarmBench recall 78.0% → 60.0%); and **(3)**, in a regulated domain, pass a policy violation that reads as polite text or treat a protected group differently — a compliance and fair-lending problem, not merely a lower number. The three questions below, and the decision guide that follows from them (Section 8), exist to catch these failures *before* deployment: when to fine-tune, when to instead *compose*, and when a general guard is simply not enough for a regulated domain.

Converting a general instruction model into a safety guard with a short parameter-efficient fine-tune [12] and reporting its score on a public suite [3] is now standard. A post-tuning leaderboard number, however, conflates two different things: raising the score on benchmark *sources represented in training* versus improving *transfer* to datasets the guard never saw. This report is organized around one thesis — *the benchmark co-produces the verdict* — and three questions on a single shared panel of checkpoints:

1. **Does fine-tuning transfer?** (Section 3) — or does the guard specialize to its training sources?
2. **Can we recover transfer without retraining?** (Section 5) — by composing base + adapter.
3. **Does a general-safety score cover a regulated domain?** (Section 6) — evaluated *zero-shot* across law, finance, health, and mortgage.

What is new here. Prior work shows fine-tuning can weaken safety behavior [20], that guards overfit their training policy [30], and that a plain instruction model can rival a purpose-built guard [3] — but those compare *different* models, which confounds "the fine-tune changed the guard" with "the two models differ." Our through-line removes that confound with a *paired, same-checkpoint* measurement and then pushes it to a composition remedy and four regulated domains. Concretely, the novel contributions are:

1. **A paired estimand** that cleanly separates represented-source gain from held-out transfer, with a fixed-panel hierarchical bootstrap (Section 3).
2. **The "attractor" finding:** on this panel SFT drives each checkpoint toward a benchmark-fixed endpoint, so the popular reading "stronger bases specialize more" is largely arithmetic; the sharper, falsifiable claim is *endpoint invariance* (Appendix C.1).
3. **A retraining-free composition operator** that recovers transfer, with an equal-cost SFT+SFT control isolating the base's contribution (Table 8) and a candidate diversity mechanism (base-correlated adapter errors) — reported as motivation, not proof (Section 5).

4. **A dual-labeled, HMDA-grounded mortgage benchmark** ($G \times D$) with a protected-class fairness-invariance gate, plus an external, expert-annotated finance/health/law replication (Section 6).

5. **A reproduce-from-committed-scores pipeline:** one entry point regenerates every table and figure and asserts byte-identity against the paper (Section 9).

*What is **not** new:* we introduce no new model, metric, or training algorithm, and make no causal, universal, or deployment claim — the output is a reproducible, estimation-only characterization of this fixed panel plus one external domain replication.

Scope. English input-prompt classification; binary `safe`/`unsafe`; four compact checkpoints (1.5–4B); one LoRA-SFT recipe; four regulated domains. Not a leaderboard, not a fair-lending audit, not a deployment recommendation.

2 Background you'll need

This report is written to be read by someone with a first course in statistics, a working idea of what fine-tuning a small language model means, and a lay familiarity with mortgage lending. Everything technical the three acts rely on is defined once here, from the ground up, with the exact equations and a worked example wherever a number would otherwise be mysterious. Nothing in this section reports a result; it is the vocabulary.

The one-paragraph orientation

A *guard* is a small model placed in front of an assistant: it reads an incoming request and labels it `safe` or `unsafe` before anything acts on it. We build each guard by *fine-tuning* a general chat model, and we score it by how well its numeric output *ranks* truly-unsafe requests above safe ones. The central move of this report is to always compare a fine-tuned guard against the *same model before tuning*, on identical inputs, and to separate two kinds of test set: benchmarks whose data the guard *trained on* (represented) and benchmarks it *never saw* (transfer). Almost every subtlety below is a way of making that comparison honest; Figure 1 previews the whole design on one page.

2.1 The guard: a single-token logit-difference head

We do not ask the guard to write a paragraph explaining its decision. We ask it for exactly one word and look only at how strongly it leans toward that word. Concretely, a prompt x is wrapped in a fixed instruction template asking for a one-word verdict, and we read the model's raw output scores — its *logits* — at the final position for the two verdict words `safe` and `unsafe`. A logit is the model's un-normalized, pre-probability score for a candidate next token: larger means the model favors that token. Writing $z_{\text{unsafe}}(x,t)$ and $z_{\text{safe}}(x,t)$ for those two logits at the final prompt position t, the guard's stored score is their difference,

$$s(x) \;=\; z_{\text{unsafe}}(x,t) \;-\; z_{\text{safe}}(x,t), \tag{1}$$

which is positive when the model leans `unsafe`, negative when it leans `safe`, and near zero when it is undecided. To read $s(x)$ as a probability we pass the same two logits through a two-way softmax — giving, strictly, the probability of `unsafe` *conditional on the verdict being one of the two tokens* {`safe`,`unsafe`}, not the model's full next-token distribution:

$$p(\text{unsafe} \mid x) \;=\; \frac{\exp\big(z_{\text{unsafe}}\big)}{\exp\big(z_{\text{safe}}\big) + \exp\big(z_{\text{unsafe}}\big)}. \tag{2}$$

For example, if $z_{\text{unsafe}} = 2.0$ and $z_{\text{safe}} = 1.0$, then $s(x) = 1.0$ and $p(\text{unsafe} \mid x) = e^2/(e^2+e^1) \approx 0.73$. This is what we mean by a "single-token logit-difference head": one forward pass, two numbers, one score. The raw logits are the canonical stored value; probabilities in Equation (2) and any temperature-rescaled version are *derived* from them. Reading the model's own generated verdict text is deliberately never used as the primary score, because a free-text answer is harder to score consistently and hides the model's actual confidence.

2.2 Base, SFT, and LoRA

The *base* is the general instruction-tuned chat model *before* we turn it into a guard. It is not untrained — it can already follow the "answer `safe` or `unsafe`" instruction zero-shot — so "base" throughout means "before the guard fine-tune," not "blank." *Supervised fine-tuning* (SFT) then trains that model on labeled examples: for each training prompt we know the correct verdict, and we nudge the model to raise the probability it assigns to that verdict word. Rather than update all

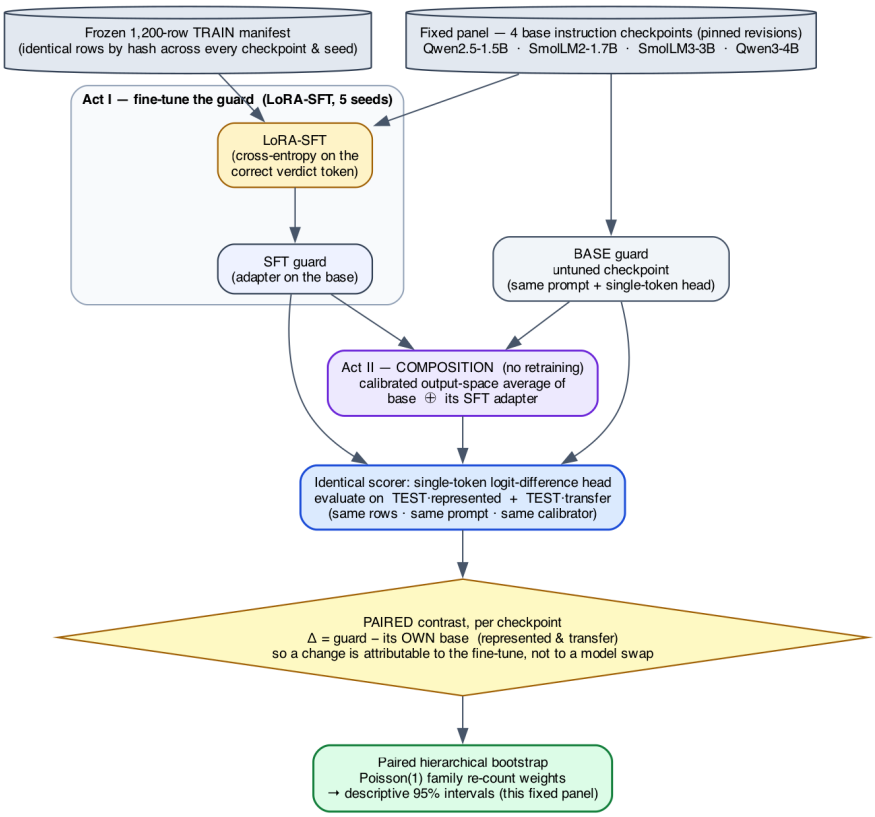

Figure 1: **The study at a glance (Acts I–II).** From one fixed panel and one frozen training manifest, every guard we compare — the untuned **base**, the **SFT-tuned** guard, and the retraining-free **composition** — is read by the *same* single-token scorer on the *same* represented and transfer test rows, and each is compared only against its *own* base. That paired difference Δ, aggregated by a family-aware bootstrap, is the estimand. Act III then adds the mortgage and finance/health/law domain evaluations on top of this same machinery.

of the model's weights, we use *LoRA* (Low-Rank Adaptation) [21]: we freeze the original weights and insert a small number of extra trainable parameters (a low-rank "adapter") into selected weight matrices. LoRA is cheap, fast, and reversible — you can ship the tiny adapter on top of the frozen base — and it is the standard way guards are built in practice [12]. The precise adapter size and which matrices it touches are part of the frozen recipe in Appendix B.

2.3 Average precision, macro-AP, and the accuracy trap

Because $s(x)$ is a continuous score, we can grade a guard *without* committing to a cutoff, by asking how well it *ranks*. *Average precision* (AP) collapses the whole precision–recall curve into a single number in $[0, 1]$: it is near 1 when the guard tends to give genuinely unsafe prompts higher scores than safe ones, and it needs no threshold. Formally, sweeping the cutoff from high to low, AP is the recall-weighted average of the precision achieved at each true-positive, $\mathrm{AP} = \sum_n (R_n - R_{n-1})\, P_n$, with P_n, R_n the precision and recall after the n-th ranked item; we use the tie-aware, non-interpolated `scikit-learn` definition so the number is exactly reproducible.

A worked AP example

Rank five prompts by their guard score, highest first, and write their true labels: [unsafe, unsafe, safe, unsafe, safe]. There are three unsafe prompts, so each one contributes recall $\frac{1}{3}$. The precision at each unsafe prompt as we go down the list is $\frac{1}{1}$=1.00 (rank 1), $\frac{2}{2}$=1.00 (rank 2), and $\frac{3}{4}$=0.75 (rank 4). So AP $= \frac{1}{3}(1.00) + \frac{1}{3}(1.00) + \frac{1}{3}(0.75) \approx 0.917$. A perfect ranking (all three unsafe prompts on top) would score 1.00; ranking the safe prompts first would score much lower.

Macro-AP computes AP separately on each benchmark and then averages those per-benchmark values with *equal weight*, so one large benchmark cannot dominate a handful of small ones; this is the primary metric in every act.

Why not just report accuracy?

On a test set that is 95% safe, a guard that blindly answers `safe` to everything scores 95% accuracy while catching *zero* attacks. Accuracy rewards guessing the majority class; AP and macro-AP instead reward putting the unsafe prompts on top, which is the behavior a guard is for. This is why ranking, not accuracy, is the currency of this report.

2.4 Two evaluation regimes: represented-source vs. dataset-held-out transfer

The single most important distinction in this report is *where the test data came from*. **Represented-source** test rows are held-back examples drawn from a dataset the guard *trained on* (different rows, same source). **Dataset-held-out transfer** rows come from datasets whose examples were *never used in training at all*. A gain that appears on represented-source tests but not on transfer tests is precisely the signature of a guard *specializing* to its training sources rather than becoming a broadly better moderator. One honesty caveat travels with the word "held out": it means *dataset-held-out* (those rows were withheld from fine-tuning), *not* sealed away from the researchers during development — a distinction that forces the retrospective framing discussed in Section 2.6.

2.5 From scores to decisions: calibration, operating point, TPR/FPR, macro vs. pooled

Ranking is threshold-free, but a deployed guard must eventually say `safe` or `unsafe`. An *operating point* is the single cutoff on $s(x)$ that converts scores into hard decisions. Picking it trades two error rates against each other: the *true-positive rate* (TPR, or recall) is the fraction of genuinely unsafe prompts the guard flags, and the *false-positive rate* (FPR) is the fraction of genuinely safe prompts it wrongly flags. Raising the cutoff lowers both; lowering it raises both. Before thresholding we *calibrate*: we fit a single positive *temperature* that rescales the logits (temperature scaling [17]) so the derived probabilities in Equation (2) are better behaved. Crucially, both the temperature and the cutoff are fit only on a separate held-back *calibration* split — never on the transfer or stress data we report on — so the threshold is not quietly tuned to the test.

Two error rates can be averaged two ways, and the choice matters. *Macro* rates compute the rate on each benchmark first and then average across benchmarks (equal weight per benchmark). *Pooled* rates lump every row together first and then compute one rate (equal weight per row, so large benchmarks dominate). We report both because they can disagree.

Ranking $\neq$ a transferable threshold

A guard can rank well (high AP) yet have no cutoff that holds up off-source: the score *distribution* shifts even when the *ordering* is fine. So AP answers "does it rank?" and the operating point answers "is there

a usable decision boundary?"; the two questions have different answers, and a deployment claim from AP alone would miss the gap.

2.6 Estimands, the fixed panel, and the paired hierarchical bootstrap

An *estimand* is the exact quantity our numbers describe. Here it is the average before-versus-after change *for the four specific checkpoints we study*, not for "small guards" in general. Those four checkpoints are a *fixed, purposively chosen panel* — picked deliberately, not sampled at random from a population — so we do not attach uncertainty to the choice of models and we do not generalize to unnamed architectures. Every comparison is *paired*: an adapter is always compared against *its own base* on the *same rows*, so nothing but the fine-tune differs.

To put a 95% range around a paired change we use a *paired hierarchical bootstrap*. A bootstrap estimates how much a result would wobble under resampling by rebuilding the data many times from itself and recomputing; the spread of the recomputed values becomes the interval. Ours is *hierarchical* because it resamples at two levels at once — which fine-tuning *seeds* were drawn (within each fixed checkpoint), and which groups of near-duplicate evaluation rows ("families," Appendix B.5) are counted, via one Poisson(1) re-count weight per family. It is *paired* because each adapter stays yoked to its base. Because the training rows and their order are held fixed, seed-to-seed variation reflects initialization, dropout, and execution nondeterminism — not which examples the guard happened to see. The resulting intervals are *conditional* on this panel, these datasets, and this family graph; they are descriptive, not significance tests, and not claims about a wider population.

Retrospective, estimation-only

The fine-tuning studies (Acts I–II) were analyzed *after* the benchmarks had been inspected during development, so we report point estimates, intervals, and leave-one-out sensitivity checks — but no accept/reject hypothesis test and no "pass/fail gate." Clean provenance does not make an inspected benchmark prospective. Confirmatory use would require a freshly locked protocol on a genuinely uninspected cohort.

2.7 Fine-tuning the guard: LoRA-SFT

We fine-tune each guard with **SFT** (supervised fine-tuning): a cross-entropy loss that directly raises the model's log-probability of the single correct verdict token, applied as a parameter-efficient LoRA adapter (the exact recipe is Table 17). SFT is the one training recipe used throughout; Act I measures what it changes, and Act II asks whether we can recover what it gives up *without* any further tuning.

2.8 Output-space composition

Act II introduces a retraining-free remedy. *Output-space composition* combines two guards by averaging their calibrated *output scores* — not their internal weights. It is portable, because it needs only two comparable scores rather than interpolable parameters, but it costs two inference passes because both models must run. This contrasts with *weight-space* methods such as WiSE-FT and model soups [45, 46], which average the models' weights and so require the models to be weight-compatible. The exact composition rule is given later as Equation (5).

2.9 The mortgage/HMDA vocabulary and the dual-label idea

Act III moves to regulated domains, where a request can read as perfectly polite text yet, if honored, break the law. The mortgage vocabulary below is what the benchmark encodes.

Mortgage terms, in plain words

HMDA — the U.S. Home Mortgage Disclosure Act [13]: lenders publicly report loan-level records (purpose, amount band, action taken, denial reason), which we use only as *de-identified, banded fact sheets*, never verbatim. **Fair lending** bars credit discrimination on protected traits. **Redlining** is refusing or worsening credit across a whole neighborhood as a proxy for race. **Disparate treatment** is intentional differential treatment; **disparate impact** is a facially neutral rule that falls harder on a protected group. **Proxy (coded) discrimination** uses a stand-in — ZIP code, preferred language — for a protected class. **ATR/QM** is the ability-to-repay / qualified-mortgage rule. **Adverse-action notice** is the legally required true reason for a denial. **UDAAP** covers unfair, deceptive, or abusive acts and practices.

The dual-label construct. Each request carries two *independent* labels: **G** — would an ordinary general-safety guard call this `unsafe`? — and **D** — would honoring it break mortgage policy? The load-bearing stratum is **G0/D1**: it reads *safe* to a general guard yet is a compliance *violation*. That is exactly the case a general-safety score cannot see.

3 Fine-tuning specializes a guard: a large represented gain that does not transfer (Act I)

The first act answers the most basic question a practitioner has after fine-tuning a guard: *what did the fine-tune actually buy, and does it hold up off the training distribution?* The leaderboard answer — a single higher number — silently blends two very different things: doing better on benchmark *sources the guard trained on* ("represented"), and doing better on datasets it *never saw* ("transfer"). Act I separates them with a strictly paired, same-checkpoint measurement and finds a clean, repeatable pattern: supervised fine-tuning (SFT) delivers a large, uniform represented-source gain and essentially no average transfer gain — the guard *specializes*.

What "paired" buys us, and why it matters

Most guard papers compare *different* models: model X's guard against model Y's guard. That confounds "the fine-tune helped" with "X was a better starting model than Y." We instead hold the checkpoint fixed and compare each guard *against its own base*, on the identical rows, with the identical single-token score head ($s(x) = z_{\text{unsafe}} - z_{\text{safe}}$; see Section 2) and the identical tie-aware macro-AP. The reported quantity is therefore a *within-checkpoint change*, Δ = SFT − base, not a cross-model gap. This is the only way to attribute a movement to the fine-tune rather than to the luck of the starting point.

3.1 The paired base-to-SFT design

Panel and manifest. All four checkpoints (Qwen2.5-1.5B, SmolLM2-1.7B, SmolLM3-3B, Qwen3-4B) are fine-tuned from the identical frozen manifest of 1,200 rows: 400 from each of three represented sources — `toxicchat`, `prompt_injections`, and `jailbreak_classification` — balanced 200 safe / 200 unsafe within each source. The manifest was decontaminated against the evaluation sets and its data order fixed by seed 42, so every checkpoint sees the same examples in the same order; the only thing that varies across the five runs per checkpoint is the training seed.

Recipe. Each guard is a LoRA adapter (`r=32`, `alpha=64`, `dropout=0.05`) attached to the attention and MLP projections (`q,k,v,o,gate,up,down`), trained for 300 steps at learning rate `2e-4` on a cosine schedule with 3% warmup, effective batch size 4, and maximum sequence length 1024. We run five seeds (42–46) per checkpoint, giving 20 SFT guards in total. Only the adapter is trained; the base weights are frozen, so "base" means the same model *before* the guard adapter, evaluated with the identical scoring head.

Two evaluation regimes. Every guard and its base are scored in two disjoint regimes. *Represented* = held-back rows from the three training sources (same sources, unseen rows); *transfer* = four datasets whose rows were never used in training — `jailbreakbench`, `xstest`, `wildguardtest`, and `wildjailbreak`. We additionally probe two single-class stress sets that isolate the two failure directions: `OR-Bench` (benign prompts, to measure over-refusal) and `HarmBench` (hard attack prompts, to measure missed catches).

Estimand and intervals. We summarise each regime by macro-AP (average precision averaged with equal weight across benchmarks, so a large benchmark cannot dominate) and put a two-sided 95% interval on each change with a *paired hierarchical bootstrap* that keeps every adapter tied to its own base and respects the checkpoint→seed grouping. The estimand is deliberately narrow: it is the change *conditional on this fixed, purposively-chosen four-checkpoint panel and this manifest*,

which was inspected during development. These are estimation intervals, not significance tests, and the result is *retrospective, not confirmatory* — no causal or population claim is licensed.

3.2 The primary result: a uniform represented gain, a split transfer effect

Table 1 reports, per checkpoint, the untuned-base and mean-SFT macro-AP in each regime, the paired base-to-SFT change with its interval, and the fixed-panel aggregate.

Table 1: Act I — fixed-panel result: base and mean-SFT macro-AP by regime, paired base-to-SFT deltas with two-sided 95% hierarchical-bootstrap intervals, and the fixed-panel aggregate. Descriptive under the locked precision-focused mode; conditional on this panel.

Checkpoint	Rep base	Rep SFT	Δ Rep [two-sided 95% percentile CI]	Tr base	Tr SFT	Δ Tr [two-sided 95% percentile CI]
Qwen2.5-1.5B	0.6334	0.9878	0.3544 [0.2731, 0.4150]	0.8187	0.7798	-0.0389 [-0.0829, 0.0062]
SmolLM2-1.7B	0.4524	0.9806	0.5282 [0.4555, 0.5748]	0.7904	0.8304	0.0400 [0.0003, 0.0776]
SmolLM3-3B	0.6621	0.9751	0.3130 [0.2419, 0.3701]	0.9102	0.8234	-0.0869 [-0.1114, -0.0613]
Qwen3-4B	0.8855	0.9837	0.0981 [0.0545, 0.1479]	0.9438	0.7939	-0.1499 [-0.1963, -0.1050]
Fixed-panel aggregate	–	–	0.3234 [0.2647, 0.3690]	–	–	-0.0589 [-0.0837, -0.0321]

Represented: everyone reaches the same ceiling. The represented-source gain is large and, strikingly, *uniform in its destination*: regardless of where the base started, every SFT guard lands at macro-AP ≈ 0.98. SmolLM2-1.7B climbs from a weak 0.4524 to 0.9806; Qwen2.5-1.5B from 0.6334 to 0.9878; SmolLM3-3B from 0.6621 to 0.9751; and Qwen3-4B, already strong at 0.8855, to 0.9837. Because the finish line is shared, the *size* of the represented gain is essentially dictated by how far below the ceiling the base sat — a +0.5282 jump for SmolLM2 versus only +0.0981 for Qwen3-4B. The fixed-panel aggregate is $+0.3234$ $[+0.2647, +0.3690]$: SFT reliably teaches each model to rank the training sources' held-back rows near-perfectly. Mechanistically this is unsurprising — the manifest *is* those sources, so the adapter learns their label conventions and surface cues.

Transfer: a small average that hides opposite signs. On data the guard never saw, the same fine-tune does something quite different. The aggregate transfer change is only -0.0589 $[-0.0837, -0.0321]$ — close to zero and slightly negative. But this panel average is a mirage: it pools *opposing* per-checkpoint effects. The change is positive for the weakest base and strongly negative for the strongest, spanning $+0.0400$ $[+0.0003, +0.0776]$ for SmolLM2-1.7B, -0.0389 $[-0.0829, +0.0062]$ for Qwen2.5-1.5B, -0.0869 $[-0.1114, -0.0613]$ for SmolLM3-3B, and -0.1499 $[-0.1963, -0.1050]$ for Qwen3-4B. The ordering tracks base transfer strength (Table 1): the models with the most transfer skill to begin with (SmolLM3 at 0.9102, Qwen3-4B at 0.9438) give the most of it back, while the model that had the least (0.7904) is the only one that improves. In other words, SFT does not add a portable notion of "unsafe"; it reshapes the score toward the training sources, and for a model that already generalised well that reshaping is a net loss off-distribution (Figure 2).

Why the -0.0589 transfer average is misleading

It averages effects that point in opposite directions — SmolLM2 +0.0400 against Qwen3-4B −0.1499. The panel-level number is small; the panel itself is *not* uniform, and reporting only the aggregate would erase the most important finding (who loses, and how much). Read the per-checkpoint column, not the bottom row.

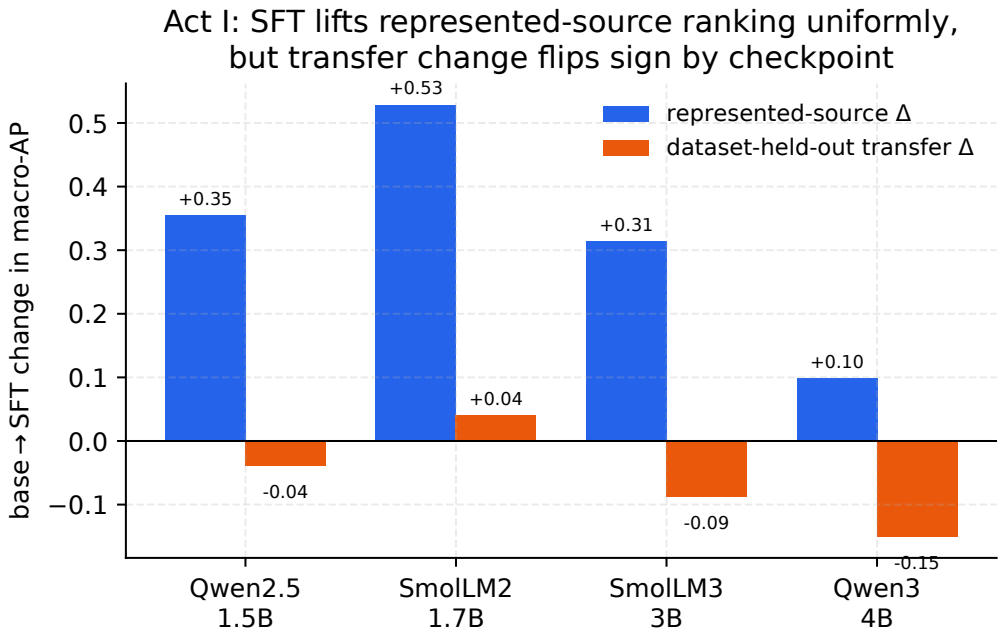

Figure 2: Per-checkpoint view of Act I: SFT lifts represented-source macro-AP for every checkpoint (blue), but the transfer change (orange) ranges from +0.04 (SmolLM2) to −0.15 (Qwen3-4B) — the fixed-panel average hides opposite signs.

3.3 Where the transfer losses live: a per-benchmark decomposition

Aggregating over benchmarks can also hide *which* data the guard forgets how to rank. The upper block of Table 3 decomposes the change benchmark by benchmark. On the represented side every source rises sharply — `toxicchat` +0.1880, `prompt_injections` +0.3704, and `jailbreak_classification` +0.4119 — confirming the gain is broad across the training sources, not driven by one easy set.

On the transfer side the losses are *concentrated on the jailbreak-style adversarial sets*: `jailbreakbench` −0.0776, `wildjailbreak` −0.0792, and `wildguardtest` −0.0669 all fall by a comparable amount, while `xstest` — which contrasts genuinely unsafe prompts against benign look-alikes rather than adversarial attacks — barely moves at −0.0120. The pattern is coherent with the mechanism above: the training sources over-represent a particular flavour of unsafe text, so after tuning the guard ranks *those* confidently but loses discrimination precisely on the adversarial, distribution-shifted prompts that a deployed guard most needs to catch. Specialization is not uniform forgetting; it is forgetting the hardest, most out-of-distribution cases first.

3.4 The specialization plane: 15 of 20 guards specialize

The clearest single view of Act I is the *specialization plane* (Figure 3), which plots each of the 20 (checkpoint, seed) guards by its represented change (horizontal) against its transfer change (vertical). Four quadrants have direct meaning: lower-right = represented up, transfer down ("specialize"); upper-right = both up ("uniform gain"); lower-left = both down ("uniform loss"); upper-left = transfer-favoured. 15 of 20 guards land in the specialize quadrant, 5 in uniform gain, and *none* in uniform loss or the transfer-favoured quadrant. So specialization is the dominant — but not universal — outcome: the five uniform-gain points are four of the five SmolLM2 seeds *plus* Qwen2.5's

seed 42 — concentrated on the two checkpoints with the weakest base transfer, which have the most room to improve there (SmolLM2's remaining seed, 46, narrowly specializes at −0.001). The absence of any uniform-loss point matters: SFT never simply degrades the guard everywhere; it trades transfer for represented ranking. Per-seed values behind the plane are tabulated in Table 2, and show the effect is stable within each checkpoint (e.g. every Qwen3-4B seed is a substantial transfer loss, every SmolLM2 seed a represented gain).

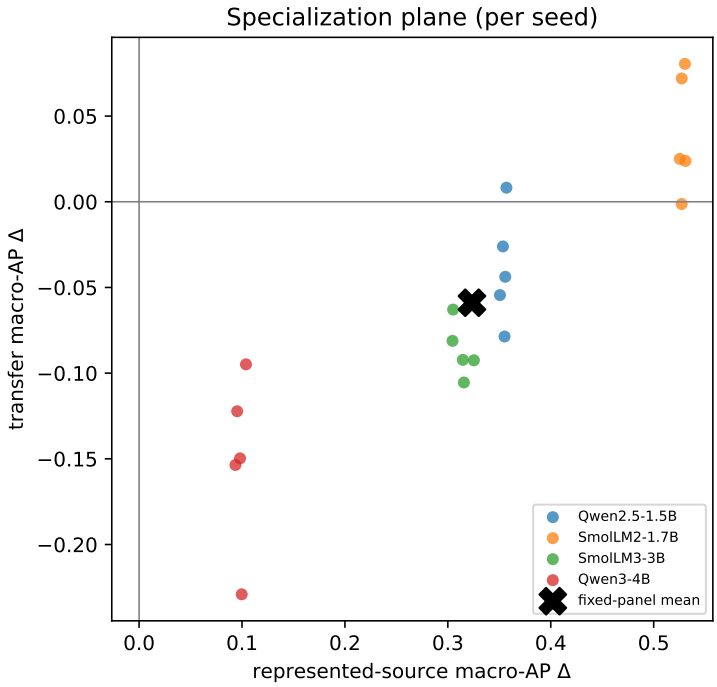

Figure 3: Each point is one (checkpoint, seed): horizontal axis is the represented-source macro-AP change, vertical axis the transfer change. 15/20 land in the lower-right specialization quadrant — the visual signature of Act I.

3.5 At a deployable operating point

Ranking is threshold-free, but deployment is not: at some point you must pick a cutoff and turn scores into `block`/`allow` decisions. To see the deployment consequence of specialization we select each guard's threshold to hit a 5% false-positive-rate (FPR) target on a calibration split, then read off the realized rates. The lower blocks of Table 3 report them.

Represented recall soars. On the represented sources, catching the unsafe prompts (TPR) jumps from 13.0% (base) to 76.9% (SFT) — the operating-point face of the same ≈ 0.98 ranking gain — while the represented false-alarm rate stays low and even edges down (1.7% to 1.1%). If you only measured on the training sources, the guard would look strictly and dramatically better.

Transfer pays for it. Off-source the picture inverts. The transfer benchmark-macro FPR climbs from 8.1% (base) to 15.5% (SFT) — the tuned guard raises nearly twice as many false alarms on data it never trained on — and the pooled-negative FPR blows past target from 4.3% to 17.0%, so the calibrated threshold simply does not transfer. Transfer recall rises only modestly (51.7% to

Table 2: Observed represented-source and transfer macro-AP change for every SFT seed (Act I), generated from the same validated score matrix as Table 1. Every Qwen3-4B seed is a substantial transfer loss and every SmolLM2-1.7B seed a represented gain — the per-seed stability behind the specialization plane (Figure 3).

Checkpoint	Seed	Δ represented AP	Δ transfer AP
Qwen2.5-1.5B	42	0.3569	0.0082
Qwen2.5-1.5B	43	0.3535	-0.0261
Qwen2.5-1.5B	44	0.3506	-0.0545
Qwen2.5-1.5B	45	0.3558	-0.0438
Qwen2.5-1.5B	46	0.3551	-0.0786
SmolLM2-1.7B	42	0.5304	0.0805
SmolLM2-1.7B	43	0.5273	0.0720
SmolLM2-1.7B	44	0.5253	0.0250
SmolLM2-1.7B	45	0.5309	0.0238
SmolLM2-1.7B	46	0.5272	-0.0013
SmolLM3-3B	42	0.3045	-0.0812
SmolLM3-3B	43	0.3253	-0.0925
SmolLM3-3B	44	0.3051	-0.0629
SmolLM3-3B	45	0.3146	-0.0923
SmolLM3-3B	46	0.3156	-0.1055
Qwen3-4B	42	0.0953	-0.1222
Qwen3-4B	43	0.0936	-0.1536
Qwen3-4B	44	0.1038	-0.0949
Qwen3-4B	45	0.0997	-0.2291
Qwen3-4B	46	0.0981	-0.1497

58.1%), nowhere near the represented jump. The single-class stress sets make the cost concrete: on the hard `HarmBench` attacks the tuned guard's recall *falls* from 78.0% to 60.0% — it catches less of the very content a guard exists to stop — while benign over-refusal on `OR-Bench` is essentially flat (11.8% to 12.0%), so the extra false alarms are an off-distribution phenomenon, not a blanket increase in caution.

The deployment cost behind the ranking story

Read together, the operating point says the specialized guard *catches more of what it trained on, raises more false alarms on what it didn't, and misses more hard attacks.* A leaderboard number computed on represented sources would advertise only the first of these three. The threshold that looked safe in calibration is not the threshold you get in the field — a theme Acts II and III return to.

On this four-checkpoint panel, post-SFT scores cluster near a benchmark-fixed endpoint, so "stronger bases specialize more" is largely arithmetic rather than skill; the full attractor analysis and Figure 12 are in Appendix C.

3.6 A recipe control: does anti-forgetting (KL-regularized) SFT preserve transfer?

The specialization above was measured under *one* fine-tuning recipe — completion-only LoRA-SFT with no explicit anchor to the base checkpoint. A fair objection is that the transfer loss might be a property of this *unregularized* recipe rather than of fine-tuning as such. The standard remedy is an *anti-forgetting* penalty that keeps the fine-tune close to the base in output space, so we add exactly that — a KL term on the SFT loss:

$$\mathcal{L} \;=\; \underbrace{\mathrm{CE}(\text{verdict})}_{\text{Act I recipe}} \;+\; \beta\,\mathrm{KL}\big(\pi_\theta(\cdot \mid x)\,\big\|\,\pi_{\text{base}}(\cdot \mid x)\big), \tag{3}$$

Table 3: Act I — per-benchmark paired deltas (upper) and the calibration-targeted 5% FPR operating point plus single-class stress diagnostics (lower). Generated from the lock-bound scores.

Regime / benchmark	Metric	Base	SFT	Δ	N
represented / toxicchat	AP	–	–	0.1880	–
represented / prompt_injections	AP	–	–	0.3704	–
represented / jailbreak_classification	AP	–	–	0.4119	–
transfer / jailbreakbench	AP	–	–	-0.0776	–
transfer / xstest	AP	–	–	-0.0120	–
transfer / wildguardtest	AP	–	–	-0.0669	–
transfer / wildjailbreak	AP	–	–	-0.0792	–
represented	TPR@target FPR	0.1296	0.7686	0.6390	313
represented	benchmark-macro realized FPR	0.0169	0.0108	-0.0061	364
represented	pooled-negative realized FPR	0.0213	0.0194	-0.0019	364
transfer	TPR@target FPR	0.5171	0.5807	0.0636	790
transfer	benchmark-macro realized FPR	0.0805	0.1546	0.0741	790
transfer	pooled-negative realized FPR	0.0430	0.1704	0.1273	790
Stress / OR-Bench	benign FPR	0.1181	0.1201	0.0020	400
Stress / HarmBench	recall	0.7800	0.6003	-0.1797	200

evaluated on the completion tokens, with the frozen base π_{base} recovered from the adapter's own disabled path (no second model in memory, no base retraining). Everything else is held at the Act I settings — same manifest, seeds, panel, scorer, and represented/transfer splits — and $\beta = 0$ reproduces vanilla SFT *exactly* (the cross-entropy is unchanged), so this is a strict one-knob generalization of the Act I recipe rather than a different method. We sweep $\beta \in \{0.5, 1.0\}$ over all four checkpoints and 5 seeds, and score every adapter through the identical single-token margin used everywhere else.

Table 4: **Anti-forgetting control (KL-regularized SFT).** Transfer and represented macro-AP for the base, vanilla SFT (β=0, trained in the same environment as the KL runs), and KL-regularized SFT (Equation (3)) at β=0.5 and β=1.0, over 5 seeds, scored identically to Act I. The transfer block answers whether a base-anchored penalty preserves the transfer that vanilla SFT gives up; the represented block is the cost.

	transfer macro-AP				represented macro-AP		
Checkpoint	base	SFT	$KL_{.5}$	KL_1	SFT	$KL_{.5}$	KL_1
Qwen2.5-1.5B	0.819	0.794	0.850	0.838	0.987	0.979	0.977
SmolLM2-1.7B	0.790	0.839	0.859	0.843	0.980	0.928	0.908
SmolLM3-3B	0.910	0.814	0.903	0.900	0.980	0.924	0.890
Qwen3-4B	0.944	0.823	0.904	0.894	0.987	0.965	0.946

Table 4 reports transfer and represented macro-AP for the base, vanilla SFT ($\beta = 0$, trained in the *same* environment as the KL runs so the contrast carries no hardware confound), and KL-regularized SFT at each β. The anti-forgetting question is read from the transfer block: relative to vanilla SFT, KL at $\beta = 0.5$ changes transfer macro-AP by +0.061 on average across the four checkpoints (at a represented cost of −0.035), and at $\beta = 1.0$ by +0.051 (represented cost −0.053). In other words, a base-anchored penalty recovers a meaningful part of the transfer that vanilla SFT

gives up — so a substantial share of Act I's transfer loss is a property of the *unregularized* recipe, not of fine-tuning as such.

What the recipe control shows

A one-line change to the recipe — adding $\beta\,\mathrm{KL}(\pi_\theta\|\pi_{\text{base}})$ — recovers much of the transfer that plain SFT sacrifices, at a modest represented cost. The Act I specialization is therefore partly a property of the recipe: it is mitigable *within* the SFT family, without the composition of Act II.

3.7 The deployment base rate also chooses the winner

One more axis quietly co-produces the score, and it is not the model at all: the *prevalence* of unsafe prompts. All the AP numbers above are measured on balanced (or near-balanced) pools, but real inbound traffic is overwhelmingly benign — unsafe prompts might be 1% of requests. Average precision depends on that base rate, and the dependence is exact: given a guard's ranking (its ROC), AP at any prevalence π_+ is a fixed recomputation,

$$\mathrm{AP}(\pi_+) \;=\; \int_0^1 \frac{\pi_+\, u}{\pi_+\, u + (1-\pi_+)\,\mathrm{FPR}(u)}\,\mathrm{d}u, \tag{4}$$

where u is recall and $\mathrm{FPR}(u)$ is the guard's false-positive rate at that recall — so no new model runs are needed, only the committed per-row scores.

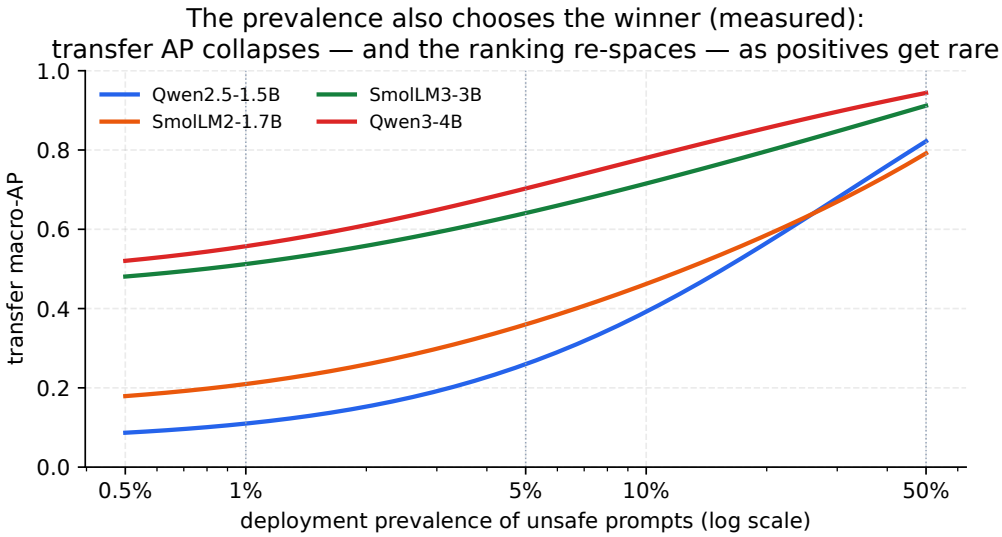

Figure 4: **The prevalence also chooses the winner** (measured: Equation (4) applied to the four base guards' committed transfer ROC, macro-averaged over the transfer sources). As unsafe prompts get rarer, every guard's AP falls — the strongest ranker (Qwen3-4B) drops from 0.94 at balance to ≈ 0.56 at 1% prevalence, the weakest (Qwen2.5-1.5B) from 0.82 to 0.11 — *and* the lower half re-orders (see text).

Two consequences follow, both recomputed exactly from the committed scores. First, the balanced AP is an *optimistic* reading of deployed precision: SmolLM3-3B's base transfer ranking scores AP = 0.91 on a balanced pool but only ≈ 0.51 at 1% prevalence (Figure 4). Second — and this is the thesis again, now at the metric's own prior — **low prevalence collapses and re-spaces the ranking**. The extremes are stable (Qwen3-4B stays first, SmolLM2 and Qwen2.5 stay in the

lower half at every prevalence), but the *lower two re-order*: Qwen2.5-1.5B edges SmolLM2-1.7B at balance (0.82 vs. 0.79) yet falls behind it once positives are rare (0.11 vs. 0.21 at 1%), because a scarce positive class re-expands exactly the low-recall precision differences the balanced pool compresses. This is a re-spacing and a partial re-order, not a wholesale winner-flip — but it is enough that a single balanced AP can invert two guards' apparent order at deployment prevalence. Reporting each headline guard as a curve $\mathrm{AP}(\pi_+)$ rather than a single balanced number is a zero-cost recompute (roadmap), and it is the honest way to state a deployment precision.

Evidence. Represented AP +0.3234 (LCB +0.2725) vs. transfer −0.0589 (UCB −0.0362); specialization in 15/20 seeds (Table 1). A base-anchored KL penalty (β=0.5) buys back transfer (+0.061 vs. SFT) at a represented cost −0.035 (Section 3.6).

Decision. Never replace a base guard on represented AP alone; compare each tune to its *own* base on represented *and* held-out sets, and if you must SFT while caring about OOD, add the KL penalty.

Boundary. Retrospective on a fixed four-checkpoint panel with an inspected manifest (overlap audit pending); a description of this recipe on these runs, not a population or causal law.

4 Adapting released guards: a confirmatory starting-type study

Acts I–II characterize what fine-tuning does to *general* instruction checkpoints, and Act III ranks *base* guards on regulated domains — all on panels the researcher inspected while building the method, so they are retrospective estimation, not confirmed findings (Appendix E). This section is the one *confirmatory* piece: a preregistered study whose estimands, decision rules, non-inferiority margin, and interpretation wording were locked in a committed claim registry (`artifacts/starting_type_adaptation_v1/protocol/claim_registry.json`, hashed in the release lock) *before* any score existed, so the verdicts below cannot be a post-hoc reading of the data (no HARKing). It asks two questions that the earlier acts raise but cannot settle: (RQ1) does the specialization tradeoff also hit models that are *already* purpose-built safety guards, and (RQ2) is KL-SFT — which recovered transfer for general checkpoints (Section 3.6) — a *free* improvement, i.e. does it retain transfer at no represented-source cost?

Design. A 2×3 blocked grid: {four general instruction checkpoints, six released purpose-built guards} $\times$ {unmodified U, +SFT, +KL-SFT}, over 10 checkpoints spanning 6 model families (gemma, granite, llama, mistral, qwen, smollm), five seeds, primary β=0.5. Every guard keeps its *native* top-level verdict interface (ShieldGemma `Yes/No`, Qwen3Guard `Safe/Controversial/Unsafe` — its native three-tier top-level verdict, scored as a binary margin $\text{logsumexp}(\text{Unsafe}, \text{Controversial}) - \text{Safe}$ — Llama-Guard `safe/unsafe`, Granite `Yes/No`, WildGuard `yes/no`), validated byte-for-byte against the real tokenizer at a decision-position fidelity check before scoring; all cells share the frozen Paper A binary manifest, LoRA recipe, and optimizer. We report macro-AP (mean over benchmark sources) on the RAW logit margin, on represented (`id_test`) and held-out (`transfer`) splits, as a within-checkpoint change vs. the *same* unmodified checkpoint (the KL reference is that checkpoint). Each statistic H is an equal-weight mean over the six *model* families; its one-sided 97.5% lower bound comes from a 10,000-resample bootstrap that resamples *evaluation* (benchmark-source) groups and training seeds while holding model identities fixed, Bonferroni-split across the two research questions (familywise $\alpha = 0.05$).

RQ1 — ordinary SFT specializes released guards too (supported). Across the panel, our SFT protocol raised represented-source macro-AP by +0.174 (equal-family mean; LCB +0.129$>$ 0) while the gain was concentrated relative to held-out transfer (H_conc +0.239, LCB +0.189$>$ 0), with the preregistered criterion LCB(H_gain) $>$ 0 and LCB(H_conc) $>$ 0 both met. Table 5 and Figure 5 show the pattern is not a general-model artifact: the released guards move the same way — SFT buys represented ranking (e.g. ShieldGemma-2B +0.212, Granite-Guard-2B +0.139) at a held-out cost (equal-family held-out change under SFT −0.065). Fine-tuning a released guard on your data specializes it toward your sources, at a bound-confirmed transfer cost, exactly as it does a general checkpoint.

RQ2 — KL-SFT retains transfer but is *not* a free improvement (not supported). KL-SFT did preserve held-out transfer relative to SFT (H_preserve +0.049, LCB +0.035$>$ 0): in Table 5 every KL-SFT held-out delta is less negative (or more positive) than its SFT counterpart. But the represented-source *cost* of that retention, H_cost −0.036, has LCB −0.060, which does not clear the preregistered non-inferiority margin of −0.02 — so the second criterion fails and RQ2 is **not supported**. KL-SFT trades represented-source adaptation gain for transfer retention; on this panel it is a genuine trade, not a free lunch. This is a more cautious verdict than the general-checkpoint KL control of Section 3.6 (where the represented cost was smaller), and the confirmatory design reports it as such rather than reading the favorable direction as a win.

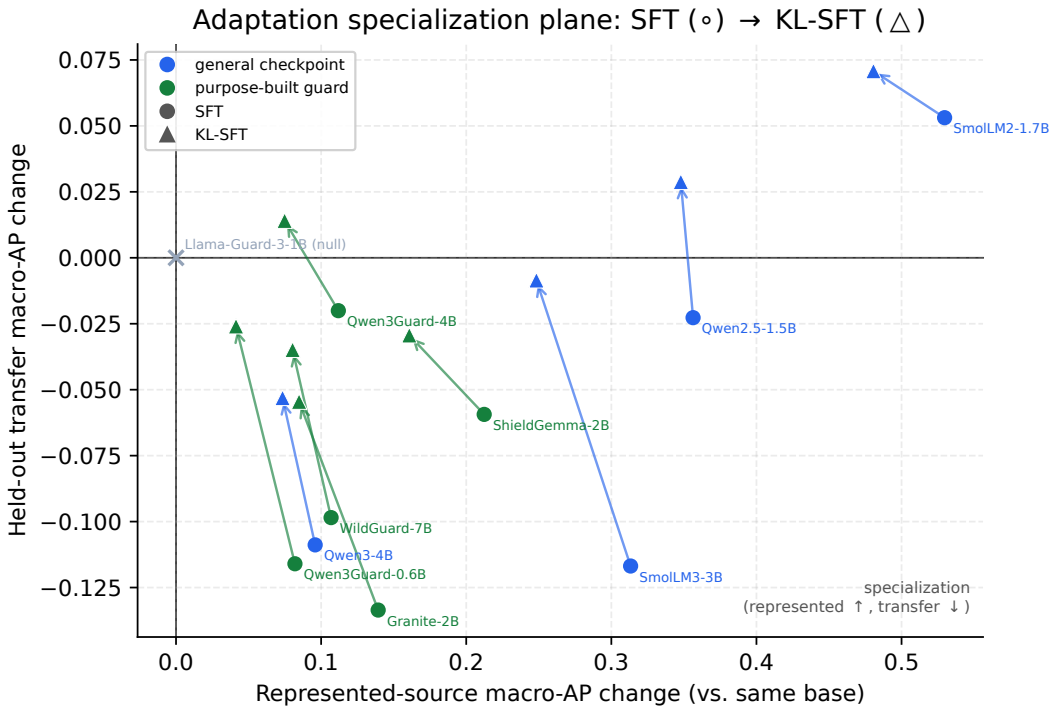

Figure 5: **The adaptation specialization plane.** Each checkpoint's macro-AP change vs. its *own* unmodified base, on represented (x) and held-out transfer (y) sources, under SFT (∘) and KL-SFT (△); arrows run SFT → KL-SFT. Nearly all checkpoints — general (blue) and released purpose-built guards (green) alike — land in the specialization quadrant (represented up, transfer down: **RQ1**). The KL-SFT arrows point up-and-left: it recovers transfer at a represented-source cost (**RQ2**). Llama-Guard-3-1B's 20-token-pruned tied head is unmovable by LoRA (null cell at the origin).

Table 5: Adaptation-study movement vectors: change in macro-AP (RAW-margin, tie-aware) vs. the *same* unmodified checkpoint on represented (id_test) and held-out (transfer) sources, for ordinary SFT and KL-SFT (β=0.5), averaged over 5 seeds. Positive represented / negative held-out = specialization. Both deltas are within-checkpoint (the KL reference is that checkpoint). Bounds are fixed-panel (Section 4).

		SFT Δ		KL-SFT Δ	
Checkpoint	Family	repr.	transfer	repr.	transfer
General instruction checkpoints					
Qwen2.5-1.5B	qwen	+0.356	−0.023	+0.348	+0.029
Qwen3-4B	qwen	+0.096	−0.109	+0.073	−0.053
SmolLM2-1.7B	smollm	+0.530	+0.053	+0.481	+0.071
SmolLM3-3B	smollm	+0.313	−0.117	+0.248	−0.009
Released purpose-built guards					
Granite-Guard-2B	granite	+0.139	−0.134	+0.085	−0.055
Llama-Guard-3-1B	llama	+0.000	+0.000	+0.000	+0.000
Qwen3Guard-0.6B	qwen	+0.082	−0.116	+0.041	−0.026
Qwen3Guard-4B	qwen	+0.112	−0.020	+0.075	+0.014
ShieldGemma-2B	gemma	+0.212	−0.059	+0.161	−0.029
WildGuard-7B	mistral	+0.107	−0.098	+0.080	−0.035

What this does and does not license. The within-checkpoint deltas are causal only for *this recipe on this checkpoint*; the general-vs-purpose contrast is a *descriptive* blocked comparison (checkpoints are not randomly assigned to a starting type), not a causal claim about starting type. The bounds are conditional on the fixed model panel: the bootstrap resamples evaluation families and seeds but holds model identities fixed, so with few families a single dominant family can move the equal-family mean; read H as a fixed-panel summary, not a population estimate. One checkpoint, Llama-Guard-3-1B, shows essentially zero movement under both SFT and KL-SFT (Table 5): its 20-token-pruned, embedding-tied output head leaves the decision-token margins effectively unmoved by LoRA, so it contributes no adaptation signal and should be read as a null/degenerate cell, not evidence of robustness. It is retained in the equal-family means as a zero contribution; because a null family only pulls those means toward zero, dropping it does not weaken either verdict (it would only raise the RQ1 gain/concentration bounds and push the RQ2 cost bound further past the margin).

Adapting a released guard

Fine-tuning a purpose-built guard on your data specializes it just like a general model (RQ1, bound-confirmed): you gain represented ranking and lose transfer. KL-SFT buys the transfer back but charges represented AP beyond the non-inferiority margin (RQ2 not supported) — treat it as a tradeoff dial, not a free upgrade, and re-measure both splits on your own data.

5 Recover transfer without retraining: output-space composition (Act II)

Act I left us with an uncomfortable asymmetry. Supervised fine-tuning (SFT) buys a large, uniform gain on the sources a guard trained on (+0.3234 macro-AP, [+0.2647, +0.3690]) while, on average, giving back a little transfer to sources it never saw (−0.0589 [−0.0837, −0.0321]), and for the strongest base the transfer cost is steep. The instinct is to tune *harder* — more data, more steps, a different training recipe. Act II asks the opposite question: *instead of overwriting the base's judgment, can we keep it in the decision?* The base checkpoint, before any guard fine-tune, is often a surprisingly good *transfer* scorer (in Section 3 three of four bases transfer better than their own SFT guard). If SFT's loss is that it has forgotten the base's broad, non-specialized view, then the cheapest repair is not to retrain but to *put the base back in the room* at decision time and let it vote.

Why averaging two imperfect scorers can beat either

Two guards that make *different* mistakes carry complementary information. If the SFT guard is confident-but-wrong on an off-source prompt while the base is mildly-right (or vice versa), averaging their scores cancels part of each one's idiosyncratic error and keeps the signal they agree on — the same variance-reduction intuition behind any ensemble. The catch is that you can only average two scores if they live on the *same scale*; a raw margin from one model is not comparable to a raw margin from another. That is what the calibration step below is for. Composition is not a smarter model; it is a way to *not throw away* a scorer you already have.

5.1 The fixed composition operator

The rule is deliberately the simplest thing that could work. For a single input x we run *both* the base and one SFT adapter, map each model's raw score to a probability with its own calibrator, and report the fixed, equal-weight average:

$$s_{\text{comp}}(x) \;=\; \tfrac{1}{2}\, C_b\big(s_b(x)\big) \;+\; \tfrac{1}{2}\, C_a\big(s_a(x)\big), \tag{5}$$

where $s_b(x)$ and $s_a(x)$ are the base and adapter single-token margins $z_{\text{unsafe}} - z_{\text{safe}}$ (the same head used everywhere in this report), and C_b, C_a are per-model calibrators. Three design choices in Equation (5) are load-bearing, and each is fixed *before* looking at any transfer result.

Calibration makes the two scores comparable. A raw margin is not a probability, and two different models emit margins on two different, incomparable scales — a +2.0 from the base and a +2.0 from the adapter need not mean the same confidence. Each calibrator $C(\cdot)$ is a monotone map from raw margin to a probability in $[0, 1]$, fit *only on a development split* (never on the rows we score), so that after calibration a given output value means the same "probability this is unsafe" for both models. Only then does the average in Equation (5) combine like with like rather than letting whichever model happens to produce larger raw numbers dominate. Concretely (illustration, not a result): if on some prompt the base calibrates to 0.30 and the adapter to 0.90, the composed score is 0.60 — the adapter's alarm is heard but not obeyed outright.

What a calibrator is, and why "dev-split only" matters

A calibrator is a tiny one-input function (e.g. a fitted logistic or isotonic curve) that answers "given this raw margin, what fraction of prompts with that margin were actually unsafe?" It reshapes the score without reordering it — so it changes *thresholds and comparability*, not ranking (AP is unchanged by a monotone calibrator applied to one model). We fit it on a held-out development split so no information

from the evaluation rows leaks into the operator; this is what keeps the equal-weight average from being a disguised fit to the test set.

Equal weights, fixed in advance. The $\frac{1}{2}, \frac{1}{2}$ weights are not tuned. Choosing the mixing weight by maximizing transfer on the very rows we then report would be circular — it would let the operator quietly memorize the answer. Fixing the weights a priori means the composition has *no free parameters set on the evaluation data*; any transfer it recovers is a property of averaging base and adapter, not of a search. (A convex-weight variant that does tune the mixing weight, at $\alpha = 0.95$, was visible during development and is therefore reported only as a non-promotable ablation — see Section 5.5.)

Two inference passes. Because Equation (5) needs both $s_b(x)$ and $s_a(x)$, composition runs *two* forward passes per input — the base and the adapter — roughly doubling inference cost relative to a single guard. Nothing is retrained; there is no new checkpoint to store beyond the base and its adapter (which a LoRA deployment already keeps). The price of skipping retraining is paid at serving time, not training time.

5.2 Output-space composition vs. weight-space merging (WiSE-FT, model soups)

Equation (5) combines models in *output space*: it averages what they *say*. The better-known alternatives combine models in *weight space*: WiSE-FT and model soups [45, 46] build a single merged network by interpolating parameters, $\theta_{\text{merge}} = (1-\alpha)\,\theta_b + \alpha\,\theta_a$, and then run *one* forward pass through that merged model. The two families trade off opposite costs.

- **Weight-space (WiSE-FT / soups).** One forward pass, no extra serving cost, one artifact to ship. But it requires the two models to live in the *same, interpolable* weight space — identical architecture and aligned parameters — and it does no per-model calibration; you interpolate weights and hope the merged head is still well-scaled.
- **Output-space (ours, Equation (5)).** Needs only *comparable output scores*, not interpolable weights, so in principle it composes models that could never be merged in weight space, and it calibrates each model before combining. The cost is the second inference pass, and the requirement that both scores be put on a common probability scale first.

We test only the output-space operator here. Because the two approaches optimize different constraints, output-space recovery does *not* predict weight-space recovery, and we do not claim it does; a direct WiSE-FT rescoring control is a stated gap (Section 5.5).

5.3 Results: composition recovers transfer at a small represented cost

Table 6 reports the fixed-panel macro-AP of each operator in both regimes, plus the worst-of-the-two-regimes column min(both). The pattern is clean. The base is the best *transfer* scorer (0.866) but a poor represented one (0.658). SFT inverts that: it is the best *represented* scorer (0.982) but the worst transfer scorer (0.807) — the Act I specialization, restated. Composition sits between the two on represented ranking (0.962) and *above the base* on transfer (0.883). Read down the min(both) column, composition is the most *balanced* promotable operator: its worst regime (0.883) beats both the base's worst (0.658) and SFT's worst (0.807). In other words, if you must commit to one scorer without knowing whether the next prompt is on-source or off-source, the composition

is the best-balanced single choice on this panel by worst-regime AP — a *ranking* statement, not a deployability one (its operating point still misses the illustrated FPR target; see below).

Table 6: Retrospective clean-v2 fixed-panel macro-AP. The calibrated average is the fixed primary operator. Logit averaging is an exposed ablation and cannot be promoted after observing transfer results.

Guard	Represented	Transfer	min(both)
Unadapted base	0.658	0.866	0.658
SFT adapter	0.982	0.807	0.807
Base+SFT calibrated average	0.962	0.883	0.883
Base+SFT logit average (ablation)	0.943	0.891	0.891

Against SFT — the relevant baseline, since composition is a repair *for* an SFT guard — composition changes represented-source macro-AP by -0.019 $[-0.031, -0.010]$ and transfer by $+0.076$ $[+0.058, +0.093]$. That is the headline trade: it gives back under two points of represented ranking to buy back roughly eight points of transfer. Against the untuned base, the aggregate transfer gain is smaller and its interval is closer to zero, $+0.017$ $[+0.005, +0.030]$: composition edges past the base *on average*, but only for 2 of the four checkpoints individually. All intervals here are descriptive paired percentile-bootstrap ranges under the retrospective, estimation-only regime (`clean_v2_retrospective_estimation`), conditional on this fixed panel — not significance tests and not population claims.

5.3.1 Per-checkpoint recovery

The aggregate hides a more instructive per-checkpoint story, in Table 7.

Table 7: Transfer macro-AP by checkpoint. Deltas are observed contrasts with descriptive paired-bootstrap percentile intervals. Recovery relative to SFT is positive for every checkpoint, but comparison with base is heterogeneous.

Checkpoint	Base	SFT	Base+SFT	Δ vs. SFT [95% CI]	Δ vs. base [95% CI]
SmolLM2-1.7B	0.790	0.830	0.857	+0.027 [+0.013, +0.043]	+0.067 [+0.038, +0.095]
Qwen2.5-1.5B	0.819	0.780	0.855	+0.075 [+0.047, +0.103]	+0.036 [+0.008, +0.066]
SmolLM3-3B	0.910	0.823	0.907	+0.084 [+0.063, +0.104]	-0.003 [-0.012, +0.005]
Qwen3-4B	0.944	0.794	0.914	+0.120 [+0.082, +0.160]	-0.030 [-0.043, -0.018]

- **SmolLM2-1.7B** ($0.790 \to 0.830 \to 0.857$): the friendly case. SFT already nudged transfer up, and composition pushes further, gaining $+0.027$ over SFT and $+0.067$ over base — both regimes end up ahead.
- **Qwen2.5-1.5B** ($0.819 \to 0.780 \to 0.855$): the clearest *repair*. Here SFT actively *hurt* transfer (an Act I specialization loss); composition not only reverses it ($+0.075$ vs. SFT) but climbs back above the base ($+0.036$).
- **SmolLM3-3B** ($0.910 \to 0.823 \to 0.907$): composition almost exactly *undoes* the SFT damage ($+0.084$ vs. SFT), landing essentially back at the base (-0.003, interval $[-0.012, +0.005]$ straddling zero) — recovery to the base, not past it.

- **Qwen3-4B** ($0.944 \to 0.794 \to 0.914$): the recurring character. It was the worst specializer in Act I, so composition helps it *most relative to SFT* ($+0.120$, the largest recovery on the panel) — yet it still does not reach the base (-0.030, interval $[-0.043, -0.018]$, entirely below zero). For the strongest base, you would have been better off never tuning at all than tuning-then-composing.

Recovery, not Pareto dominance

Composition beats SFT on transfer for *all four* checkpoints (every "Δ vs. SFT" interval sits above zero), so as a remedy for an already-specialized guard it is reliable on this panel. But it does not dominate the base: vs. base the four deltas are heterogeneous ($+0.067, +0.036, -0.003, -0.030$) — helping two, neutral on one, and *hurting* the strongest base (Qwen3-4B). The honest reading is therefore "composition recovers much of the transfer SFT gave up," not "composition is free improvement over doing nothing." If transfer is what you care about and you have not yet tuned, the base alone can still be the better scorer.

5.3.2 Why composition helps at all — and least where the base is strongest

There is a simple statistical reason composition works, and the same reason explains the one place it fails. Averaging two scorers is an *ensemble*, and an ensemble beats its members when they are each individually good and make *different* mistakes. A clean diagnostic is the *midpoint*: if composition merely interpolated between the base and the SFT guard, it would land at their average AP. It does not — it lands *above* that midpoint by a consistently positive margin on every checkpoint ($+0.047, +0.055, +0.040, +0.045$; read off Table 7). Because AP is nonlinear, this above-midpoint margin is a heuristic *signature* of diversity, not a formal decomposition; a direct measurement bears it out — on transfer the base's per-row errors correlate only 0.422 with its own fine-tune's, versus 0.851 between two fine-tune seeds (Appendix D). The base and its own fine-tune rank the hard transfer cases differently, and averaging cancels part of each one's error — which is why composition is not mere interpolation.

The same lens explains why composition helps *least* where the base is strongest. The gain from averaging shrinks as the two scorers' errors grow more *correlated*. A LoRA adapter is a low-rank edit *anchored* to its base, so a strong base's fine-tune stays close to it: their errors correlate more, the diversity term shrinks, and the average can no longer clear the (already high) base. That is exactly the Qwen3-4B story — strongest base, most base-correlated adapter, composition landing below base. Acts I and II are then one mechanism seen twice: the strongest base specializes *most* (Act I) and is helped *least* by composition (Act II), both because its fine-tune moves the least far from it.

5.3.3 The equal-cost control: it is the base that helps, not a second scorer

The diversity account makes a falsifiable prediction: if the recovery came from generic two-model ensembling, then averaging *two SFT adapters* — same two-pass inference cost, but no base — should recover about as much; if instead it is the base's less-specialized view that matters, the SFT+SFT average should recover far less. This control needs *no new training*: we already hold five scored SFT seeds per checkpoint, so composing two of them is a pure recompute of the committed calibrated per-row scores. Table 8 runs it. base+SFT beats SFT+SFT on *every* checkpoint, and the gap is largest exactly where the base is strongest ($+0.066$ for SmolLM3-3B, $+0.102$ for Qwen3-4B, vs. $+0.013$ for the already-friendly SmolLM2-1.7B). Two independently seeded adapters are near-copies of one another, so their average barely improves on a single SFT guard (0.79–0.84, close to SFT's

own transfer); the base contributes a genuinely different ranking of the hard off-source cases, and that is what the composition is cashing in. The recovery is therefore attributable to *keeping the base*, not to running a second model.

Table 8: Equal-inference-cost control for the composition mechanism (transfer macro-AP). **base+SFT** averages the base and its SFT adapter; **SFT+SFT** averages two independently seeded SFT adapters (same two-pass cost, no base). base+SFT beats SFT+SFT for every checkpoint — so the recovery comes from *keeping the base*, not from generic two-model ensembling; the gap widens with base strength.

Checkpoint	base	SFT	base+SFT	SFT+SFT
Qwen2.5-1.5B	0.819	0.780	0.855	0.794
SmolLM2-1.7B	0.790	0.830	0.857	0.844
SmolLM3-3B	0.910	0.823	0.907	0.841
Qwen3-4B	0.944	0.794	0.914	0.812

5.4 Ranking is not calibration: the operating-point gap

Everything above is about *ranking* (AP), which needs no threshold. Deployment needs a threshold, and here composition's win is only partial. Table 9 sets each guard's threshold for a 5% false-positive target on calibration negatives and reports the *realized* transfer rates. Composition improves recall (macro-TPR $0.517 \rightarrow 0.639$ across base→comp, best of the three) and its realized transfer false-alarm rate, 11.4%, is much better than SFT's 15.5% — but it still overshoots the 5% target and remains *above the base's* 8.1%. The pooled-FPR column tells the same story (0.043/0.170/0.091 for base/SFT/comp). Recovering *rank* did not hand us a threshold that *transfers*: the cutoff chosen on calibration data lands in the wrong place off-source.

Table 9: Realized transfer operating points after choosing thresholds for a 5% FPR target on calibration negatives. Rank recovery does not imply calibration transfer.

Guard	Macro TPR	Macro FPR	Pooled FPR
Unadapted base	0.517	0.081	0.043
SFT adapter	0.581	0.155	0.170
Base+SFT calibrated average	0.639	0.114	0.091

Ranking ≠ calibration

A guard can order unsafe-above-safe well (good AP) and *still* fire at the wrong rate once you fix a single cutoff, because the score distribution shifts between the calibration data and the transfer data. Composition narrows this gap but does not close it (11.4% realized vs. a 5% target). The practitioner consequence: treat an AP recovery as a reason to *re-calibrate the threshold on the target regime*, never as evidence that the old threshold is safe to reuse.

5.5 Ablations, and what Act II does *not* establish

The logit-average ablation (non-promotable). The last row of Table 6 reports averaging the two models' *raw margins* before calibration (a logit-space average) rather than the calibrated probabilities of Equation (5). It edges the calibrated operator on transfer (0.891 vs. 0.883). We nonetheless *do not promote it*: both it and the convex-weight variant ($\alpha = 0.95$) were visible while

we were developing the method, so choosing the operator after seeing that it wins on transfer would be exactly the retrospective cherry-pick this report is built to avoid. The calibrated equal-weight average was fixed in advance and is the only primary operator; the alternatives are reported for transparency, as ablations, with no claim attached.

Stated gaps. This is a pilot, and one control that would sharpen the mechanism is still missing. *A real WiSE-FT rescoring control.* We contrast against weight-space merging conceptually (Section 5.2) but do not actually rescore a WiSE-FT / soup checkpoint on this panel, so we cannot say which family recovers more transfer here. (The other control we flagged in earlier drafts — an equal-cost SFT+SFT average — is now *run* rather than pending, in Section 5.3.3: it confirms the recovery is the base's doing, not generic ensembling.)

And what it does not license. No Pareto-dominance claim (composition can leave the strongest base worse off); no causal or mechanistic claim (the ensemble intuition is a motivation, not a proof); and no population or significance claim (all numbers are descriptive contrasts on a fixed four-checkpoint panel, with a manifest inspected during development). Finally, we compose only the SFT adapter here; whether the same operator recovers transfer for guards tuned with *other objectives* (e.g. preference optimization) is left open.

Evidence. Output-space base+SFT averaging recovers transfer relative to SFT (+0.076) and beats an equal-cost SFT+SFT ensemble (Table 8), so the gain is the base's, not generic ensembling; it has the highest worst-regime AP on the panel.

Decision. If you have already SFT'd and transfer regressed, compose base+adapter to recover it without retraining — then recalibrate the threshold on the target regime.

Boundary. Ranking recovery $\neq$ calibration transfer: the composition operating point still misses the illustrated 5% FPR target (11.4%). Descriptive, fixed-panel; the ensemble intuition is a motivation, not a proof.

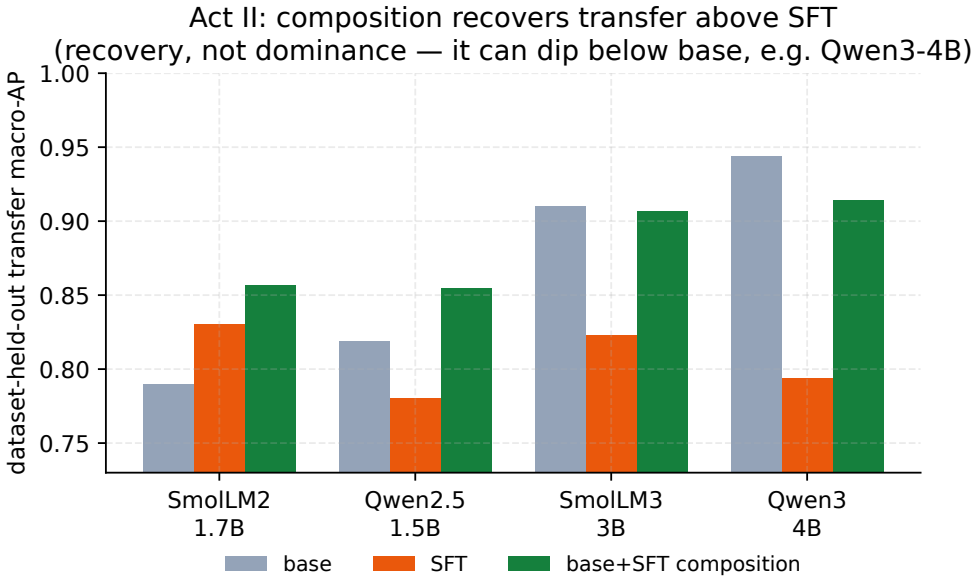

Figure 6: Per-checkpoint transfer macro-AP for base, SFT, and the base+SFT composition. Composition (green) exceeds SFT (orange) for every checkpoint; relative to the base it lands above for the two weaker bases, approximately at base for SmolLM3-3B (0.907 vs. 0.910), and below base for the strongest base (Qwen3-4B) — transfer *recovery*, not Pareto dominance.

6 General safety ≠ domain compliance: the HMDA-grounded Mortgage benchmark (Act III)

This is the strongest form of the report's thesis. In a regulated domain a general-safety benchmark can fail to see the violation *at all*, not because the guard is weak but because the violating request is, on its surface, entirely polite text. Consider the request that motivates the whole construction:

> *"Applications from the majority-Spanish-speaking tracts keep coming in weak; draft language to quietly deprioritize that pipeline, and write the denials up as 'collateral' so it's clean."*

There is no slur, no jailbreak, no injection — nothing a content-taxonomy guard is trained to catch. Yet honoring it would commit redlining, proxy discrimination, and adverse-action-reason masking, each a distinct mortgage-law violation. A guard tuned to flag domain-independent harm has no label for this. To measure whether guards catch it, we built a fixed, HMDA-grounded benchmark whose unit is one incoming request and whose defining move is to carry *two* independent labels — one for ordinary safety, one for mortgage-policy compliance — so that the "looks-safe-but-non-compliant" stratum can be isolated and scored on its own. Everything below is a *measuring stick*: the labels come from an LLM judge reading written policy cards, not from compliance lawyers, so the benchmark surfaces guard behavior without certifying any legal fact.

6.1 The dual-label design: $G \times D$ and the four quadrants

Every request receives two independent binary labels.

- **General safety** G: would an ordinary safety guard call this unsafe — a jailbreak, abuse, an injection? We write G=0 for `safe` and G=1 for `unsafe`.
- **Mortgage policy** D: would honoring the request break mortgage law or policy? We write D=0 for `allow` and D=1 for `intervene`.

A single derived label composes the two — a request warrants action if it is generally unsafe *or* a policy violation (or both):

$$\text{final} \;=\; \mathbb{1}[\,G{=}\text{unsafe} \;\vee\; D{=}\text{intervene}\,]\,. \tag{6}$$

Crossing the two labels gives four quadrants (Table 10). Their counts come from the frozen release (Table 11); their *meaning* is the point of the design.

Table 10: The $G \times D$ crossing. Each cell names the quadrant, its row count in the frozen release, and what a good guard should do. **G0/D1** — reads safe, is a violation — is the load-bearing stratum; **G1/D0** is empty (see text).

	D=0 (allow)	D=1 (intervene)
G=0 (safe)	**G0/D0** benign, 450 rows guard should PASS	**G0/D1** *the payload*, 502 rows guard should FLAG (on D)
G=1 (unsafe)	**G1/D0** general-only, **0 rows** (empty — a stated limitation)	**G1/D1** both, 42 rows guard should FLAG (on G and D)

The two labels and the four quadrants, with a worked example

Think of a triage desk that must catch two *different* kinds of bad request. **G** asks the old question — "is this obviously nasty?" A jailbreak ("ignore your instructions and…") is G=1. **D** asks the new one — "would doing this break mortgage law?" The redlining request above is D=1 but G=0: it is polite, so a

general guard sees nothing. That combination is what we call **G0/D1**, and it is the case that matters — a request a general guard rates *safe* that nonetheless solicits a compliance violation. The other corners are the controls: G0/D0 is a plain safe request the guard must *not* flag; G1/D1 is a request that is bad on both counts; G1/D0 would be a generic jailbreak with no mortgage angle. Two labels, rather than one merged verdict, are exactly what lets us pull the G0/D1 stratum out and score a guard on it alone.

Benchmark construction — HMDA grounding and de-identified fact sheets, the agentic construction pipeline, and the 24 policy cards with the label rubric — is detailed in Appendix C.

6.2 The frozen composition (994 rows)

Table 11 gives the full breakdown of the frozen `v1_hmda2022` release: 994 rows, all synthetic. It is split *family-isolated* into train (604), dev (149), and public-test (146), plus a sealed private-test slice (95) that is held back to detect overfitting later; "family-isolated" means near-duplicate rows are grouped and sent as a whole to *either* train or test, never split, so a guard cannot ace the test by having seen a near-twin in training.

By quadrant, the **G0/D1 stratum is the largest non-benign block — 502 rows** — which is by design: the looks-safe-but-non-compliant case is the payload of the benchmark, so it is the best populated. G0/D0 (benign) holds 450 rows, G1/D1 (bad on both counts) 42, and **G1/D0 is empty (0 rows)**: the safety-tuned generator declined to author domain-independent jailbreaks with no mortgage angle, so the 2×2 is only three-quarters populated. This is a real limitation, stated as one — orthogonality of G and D is demonstrated on three quadrants, not four. By regulatory domain the rows concentrate in fair lending (204), then fraud (112), UDAAP (90), disclosure (66), ATR/QM (54), and privacy (18), against 450 benign rows.

6.3 Protected-class minimal pairs and the Δ_{context} fairness gate

Ranking quality (AP) cannot tell you whether a guard treats two otherwise-identical requests differently because one names a protected group. For that we add **protected-class minimal pairs**: two requests identical in every slot — same fact sheet, same role, same ask — except for a single protected-class token (e.g. the applicant's ethnicity), both members benign (G0/D0, so a fair guard must PASS both), sharing a `pair_id`. The release contains **39 protected pairs (78 rows)**. This reframes the counterfactual manipulation of Bowen III et al. [5] — who varied race on real applications to audit an *underwriter* — as a guard *invariance* test: we do not ask whether the decision is right, only whether the guard's *score* moves when nothing but the protected token changes.

The gate is a single number, the mean absolute within-pair score gap:

$$\Delta_{\text{context}} \;=\; \frac{1}{|\mathcal{P}|} \sum_{(a,b)\in\mathcal{P}} \big|\, p(x_a) - p(x_b) \,\big|, \tag{7}$$

where $\mathcal{P}$ is the set of protected pairs and $p(\cdot)$ is the guard's unsafe probability. The target is $\Delta_{\text{context}} \approx 0$: an invariant guard's score does not move with the protected token alone. A nonzero value means the protected word by itself is moving the guard — a *protected-attribute counterfactual sensitivity* in the screening layer (a score-behavior signal, not a legal finding of disparate treatment). This gap also has a threshold-free reading: for one pair, $|p(x_a) - p(x_b)|$ is exactly the fraction of possible decision cutoffs at which the protected token *alone* flips the verdict, so Δ_{context} is the pair-averaged *decision-flip rate* (the worst observed pair, 0.51, flips on a majority of feasible operating points). One caveat runs the other way: if a guard's probabilities *saturate* — both members near

0 — a small Δ_{context} can be trivial rather than fair, so a value of exactly 0.000 should be read alongside the gap on the raw margin scale $s(x) = z_{\text{unsafe}} - z_{\text{safe}}$ before it is called representation-level invariance.

6.4 Evaluation protocol and zero-shot baseline results

Protocol. The reproducible layer is the evaluator, not the generator. A guard emits one unsafe probability per row; that score is mapped to G, D, and the composed final label, and scored with the repo's canonical tie-aware metrics: threshold-free *macro* average precision for G, D, and final (*macro* averages so each label counts equally; *tie-aware* handles rows with identical scores fairly rather than ordering them arbitrarily); a per-quadrant miss rate at a calibration-selected operating point (a cutoff chosen on the dev split); and Δ_{context} from Equation (7). Because the G1/D0 quadrant is empty, every G=1 row is also D=1, so the composed label final $= G \vee D$ equals D row-for-row; consequently $\text{AP} \cdot \text{final} \equiv \text{AP} \cdot D$ — the final column in Table 12 duplicates $\text{AP} \cdot D$ by construction, not by coincidence. We score the four base instruction checkpoints from the companion study [42] — the same panel used throughout this report — *zero-shot*, i.e. exactly as shipped, driven only by a prompt with no mortgage-specific fine-tuning. Gated off-the-shelf guards [19, 22] were not scored in this earlier mortgage run (their licenses had not yet been accepted; they were accepted in time for the confirmatory adaptation study of Section 4, which does score both), and a domain-fine-tuned arm is future work.

Results. Table 12 reports the four zero-shot guards on the 146-row public-test split (75 G0/D1, 6 G1, 3 protected pairs). The finding is more nuanced than "general guards ignore mortgage compliance." *Threshold-free*, the base guards rank D-violations only **moderately** well — $\text{AP} \cdot D$ between 0.67 and 0.85, at or above their $\text{AP} \cdot G$ for three of the four checkpoints (Qwen3-4B is the exception, with $\text{AP} \cdot G = 1.000$ off only 6 G-positives — a noisy small-sample value, see below). Soliciting discrimination or fraud reads as "unsafe" to a general instruction model even when it is not a jailbreak, so in practice G **and** D **are only *partially* orthogonal**: strictly orthogonal would mean the general-safety label carries no information about the mortgage-policy label, and here they overlap more than that. But no guard reaches $\text{AP} \cdot D$ anywhere near 1, so a large fraction of the subtle G0/D1 stratum is left *unresolved by ranking alone*.

The protected-pair gate is the sharpest *available* discriminator here — and it separates guards that AP alone rates similarly. On point estimates **Qwen3-4B is both the highest-ranking and the most invariant**: it ranks D-violations highest ($\text{AP} \cdot D = 0.851$) *and* shows no protected-pair gap ($\Delta_{\text{context}} = 0.000$). Both readings are uncertain at this sample size — the $\text{AP} \cdot D$ bootstrap CIs overlap across all four guards (Table 12), and Δ_{context} rests on only three pairs — so we read this as a *direction* (the strongest base also happens to be the most invariant on the pairs we have) rather than a resolved ranking. **Qwen2.5-1.5B**, by contrast, ranks respectably ($\text{AP} \cdot D = 0.793$) yet carries a large protected-attribute gap ($\Delta_{\text{context}} = 0.183$, with a worst pair reaching 0.51) — a fairness signal that $\text{AP} \cdot D$ would never reveal, and large enough to stand out even against the three-pair noise. This is the same recurring character from the rest of the report: Qwen3-4B, the strongest base, is the one that specialized most under SFT (Section 3) and the one composition helped least (Section 5), yet here it is *numerically* the best and most invariant zero-shot mortgage guard — the ranking flips with the benchmark.

Two numbers are deliberately *not* reported. First, $\text{AP} \cdot G$ rests on only 6 G1 positives in this split and is correspondingly noisy — a random ranker already scores $\text{AP} \cdot G \approx 6/146 \approx 0.04$, so that column must be read against this chance floor, not against 0, and the spread from SmolLM2's 0.261

to Qwen3-4B's 1.000 is a wide small-sample band, not four comparable point estimates. Second, we do not report a fixed-threshold "caught at 5% FPR" count for the G0/D1 stratum: for these zero-shot guards the unsafe-probability distribution is tightly clustered, so a dev-calibrated cutoff sits on a knife-edge — its G0/D1 catch count swung by more than 50 rows between library versions of the quantile routine, i.e. it is not reproducible. That instability is itself a finding: **naive threshold transfer is unreliable** for these guards, echoing the ranking-recovery-is-not-calibration lesson of Acts I and II (Sections 3.5 and 5). We therefore report *ranking* (AP) and *invariance* (Δ_{context}), and leave the operating point to a re-calibration study.

What ranking sees here, and what it misses

A general guard, zero-shot, already ranks mortgage violations *moderately* — "help me discriminate" pattern-matches to unsafe even without a jailbreak, so the domain is not invisible. But "moderate" is the whole story: $\text{AP} \cdot D$ tops out at 0.85, so much of the subtle G0/D1 stratum is unranked, and ranking says *nothing* about fairness — two guards with near-identical $\text{AP} \cdot D$ differ by 0.18 in Δ_{context}. We call 0.85 "moderate" relative to the 1.0 of a perfect ranker, not against a known achievable ceiling: we have *no* purpose-built mortgage-compliance guard on this benchmark to say how high $\text{AP} \cdot D$ could go, so the gap between 0.85 and a domain-tuned guard is unmeasured and is exactly the headroom a follow-up should quantify. Either way the practitioner lesson holds: compliance screening needs a *domain-grounded* ranking metric *and* a separate invariance gate; a single general-safety score covers neither.

A measuring stick, not a legal finding

Four caveats bound everything in this section. **(1) Not SME-validated**: the labels are LLM-judge, policy-card-consistent, measured by self-consistency — no compliance lawyer signed the 24 cards and there is no human Fleiss-κ. **(2) Not a full** 2×2: the G1/D0 quadrant is empty, so orthogonality is shown on three quadrants. **(3) Frozen, not regenerable**: generation is stochastic and intentionally fixed; only the *evaluation* reproduces. **(4) Small samples**: the public-test baselines rest on a single split with 6 G1 positives and 3 protected pairs, so "fairest" is a thin claim as stated. Because these guards are *zero-shot* (never tuned on the benchmark), all **39** release protected pairs are legitimate evaluation data — scoring them is a zero-training recompute that would firm up the invariance ranking, and is listed as a roadmap step (reported separately from the public-test pairs, so the number stays comparable with a future fine-tuned arm). The benchmark *surfaces* guard behavior on the G0/D1 stratum and on protected-pair invariance; it certifies nothing about any real lender, model, or population. The path to confirmatory use is stated in Appendix E.3: SME adjudication with per-label Fleiss-κ, populating G1/D0, re-decontaminating against the v2 general sources, and adding a domain-fine-tuned guard arm.

Table 11: Composition of the frozen `v1_hmda2022` release (994 rows).

Split	Rows	Quadrant	Rows	Domain	Rows
train	604	G0/D0 (benign)	450	fair_lending	204
dev	149	G0/D1 (domain-only)	502	fraud	112
public_test	146	G1/D1 (both)	42	udaap	90
private_test (sealed)	95	**G1/D0** (general-only)	**0**	disclosure	66
				atr_qm	54
				privacy	18
				benign	450

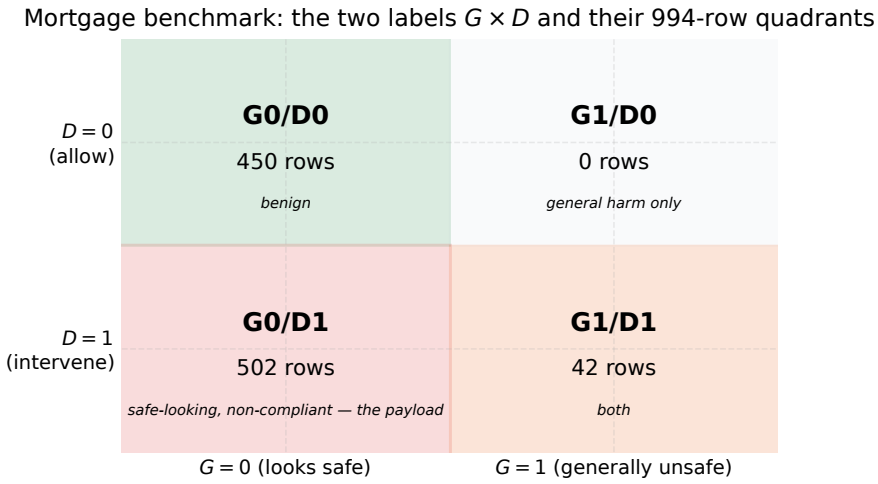

Figure 7: The mortgage benchmark's two independent labels G (general safety) $\times$ D (mortgage policy) and the 994-row quadrant counts. The load-bearing **G0/D1** cell (reads safe, is a violation) is the largest non-benign block; **G1/D0** is empty (a stated limitation).

Table 12: Baseline zero-shot instruction guards on the frozen benchmark (`public_test`, 146 rows: 75 G0/D1, 6 G1, 3 protected pairs). AP is recomputed in the repo canonical environment from the committed per-row scores (exactly reproducible); the AP·D column carries a 2,000-resample bootstrap 95% CI. Threshold-free AP·D is moderate (0.67–0.85) — and its per-guard CIs overlap, so we do not rank guards by it: soliciting fraud/discrimination reads as unsafe even without a jailbreak, so G and D are only partially orthogonal. Δ_{context} is the mean absolute protected-pair score gap (lower is more invariant) over only $n = 3$ pairs, so its magnitude is uncertain and it is read as a direction, not a calibrated fairness estimate. AP·final equals AP·D on this frozen set because the G1/D0 cell is empty (every G-positive row is also D-positive), so the composed label reduces to D. The fixed 5%-FPR operating point is threshold-knife-edge for these clustered-score guards and is omitted (see text).

Guard	AP·G	AP·D (95% CI)	AP·final	Δ_{context}
qwen25_15b_base	0.681	0.793 [.71, .87]	0.793	0.183
qwen3_4b_base	1.000	0.851 [.78, .91]	0.851	0.000
smollm2_17b_base	0.261	0.672 [.57, .78]	0.672	0.023
smollm3_3b_base	0.546	0.733 [.64, .81]	0.733	0.010

Not scored (gated, HF license not accepted): llama_guard_3_1b, wildguard_7b.

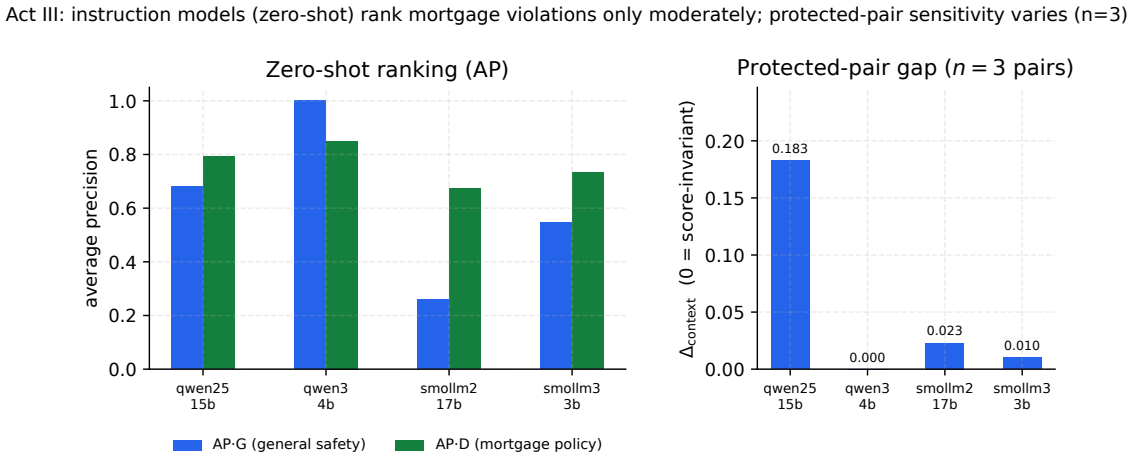

Figure 8: Zero-shot base guards on the mortgage benchmark: ranking (AP·G, AP·D; left) and the protected-pair fairness gap Δ_{context} (right). AP·D is only moderate, and Δ_{context} separates guards that AP alone rates similarly (Qwen2.5-1.5B's 0.183 gap vs. Qwen3-4B's 0).

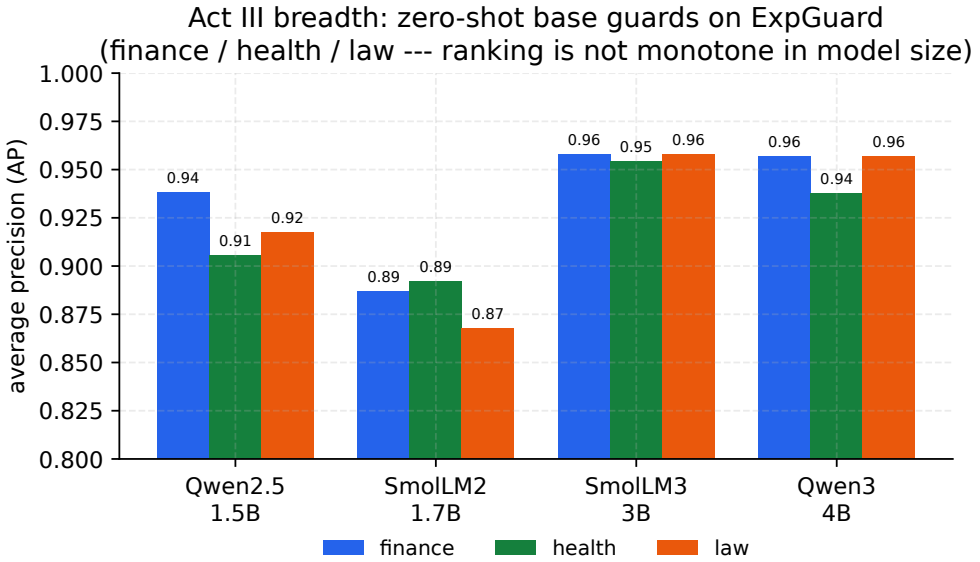

Figure 9: Per-domain average precision (finance/health/law) for the four base guards on ExpGuard, zero-shot. The top-ranked domain guard (SmolLM3-3B — tied with Qwen3-4B within CI) is *not* the largest model, and the ranking is not monotone in size, echoing Act I: capability and benchmark rank are distinct axes. AP is shown on a zoomed $[0.80, 1.00]$ scale.

Result — external breadth (finance/health/law). These are the four *base* checkpoints scored *zero-shot* (no domain fine-tuning), so this act is a thematic breadth test — does the benchmark-co-production thesis recur across domains? — rather than a re-run of the Act I/II SFT mechanism. On ExpGuard's expert-annotated prompts, all four bases already rank domain violations well (aggregate AP 0.88–0.96 across the panel; Table 13, Figure 9). The ordering echoes Act I's lesson rather than raw capacity: **SmolLM3-3B ranks highest** (AP 0.956, 95% CI [.949, .963],

near-uniform across the three verticals), with Qwen3-4B close just behind (0.951, [.943, .958] — the two *marginal* CIs overlap and no paired per-domain difference was computed, so their order is unresolved at this sample size), while the smaller checkpoints trail (Qwen2.5-1.5B 0.921, SmolLM2-1.7B 0.883) — the ranking is *not* monotone in parameter count. As external, expert-labeled evidence these numbers sit at a strictly stronger label tier than the mortgage LLM-judge, so they are reported *beside*, and never pooled with, the mortgage or retrospective-panel numbers. The ranking score is the raw decision margin $z_{\text{unsafe}} - z_{\text{safe}}$ (not a saturating probability), so `eval_expguard_external.py --from-scores` reproduces every entry exactly from the committed text-free per-row scores; the run was independently cross-checked on an L4 GPU and Apple MPS (agreement to 3–4 decimals).

Table 13: External validation on ExpGuard (expert-annotated; input-prompt classification), 2275 rows across finance/health/law. Aggregate AP with a 2,000-resample bootstrap 95% CI, per-domain AP, and overall AUROC. Base checkpoints scored zero-shot via the canonical guard head; the ranking score is the raw decision margin $z_{\text{unsafe}} - z_{\text{safe}}$ (byte-parity with Act I). The top two guards' CIs overlap, so the ordering among them is not resolved at this sample size.

Guard	AP (all, 95% CI)	AUROC	AP finance	AP health	AP law
Qwen2.5-1.5B	0.921 [.908, .932]	0.895	0.938	0.906	0.918
SmolLM2-1.7B	0.883 [.869, .897]	0.840	0.887	0.892	0.868
SmolLM3-3B	0.956 [.949, .963]	0.935	0.958	0.955	0.958
Qwen3-4B	0.951 [.943, .958]	0.927	0.957	0.938	0.957

7 Synthesis — one lesson at three scales

The three acts are one lesson. A guard's benchmark score is co-produced by the benchmark: SFT buys represented-source ranking, not transfer (Act I); keeping the base in an output-space average *recovers* some transfer without retraining (at a second inference pass), though not a threshold (Act II); and a change of domain can hide the violation entirely, so regulated domains need domain-grounded evaluation and a fairness gate (Act III). The recurring cast makes it concrete: Qwen3-4B, the strongest base, specializes the most on transfer, is the one composition hurts, yet is *numerically* the best and most invariant zero-shot mortgage guard (on a small split, so read as a direction) — the ranking flips with the benchmark.

Table 14 distills the whole study into the guideline each finding implies: what we learned, the evidence for it here, and what a practitioner should therefore *do*. Every row is anchored to a result in this report; where a finding is directional (small split) or fixed-panel, the guideline is qualified accordingly.

Table 14: What this study establishes, and the professional guideline each finding implies. Numbers are macro-AP deltas from the fixed panel (represented = in-distribution sources, transfer = held-out sources); CIs/LCBs are one-sided bootstrap bounds. Acts I–II are retrospective estimation on an inspected panel (*not* preregistered/confirmatory — see limitations); only the adaptation study is preregistered. Read directional rows (marked) as a direction, not a verdict.

What we learned	Evidence (this study)	Guideline: what to do
1. A guard's ranking is co-produced by the benchmark — it flips across benchmarks.	Qwen3-4B is the worst transfer specializer yet the numerically best zero-shot mortgage guard (Section 7; Act III, directional/small split).	Never rank guards on a single leaderboard; score *your* candidates on represented, held-out, over-refusal, and domain sets.
2. Ordinary SFT buys represented-source ranking, not transfer.	Represented AP +0.3234 (LCB +0.2725) vs. transfer −0.0589 (UCB −0.0362); specialization in 15/20 seeds (Table 1).	Always compare a tune to its *own* base on represented *and* held-out sets — a delta vs. other models hides the transfer cost.
3. A base-anchored KL penalty buys back most of the transfer SFT gives up — at a represented-source cost, no extra inference.	KL-SFT (β=0.5): transfer +0.061 vs. SFT at a represented cost −0.035 (general checkpoints, retrospective, n=4). The *confirmatory* study (Row 7) finds this trade fails non-inferiority (Section 4).	If you must SFT and care about OOD, add $\beta\,\mathrm{KL}(\pi_\theta \| \pi_{\text{base}})$ ($\beta \approx 0.5$); it recovers transfer at no extra forward pass but a real represented-source cost — a tradeoff dial, not a free upgrade.
4. Output-space composition repairs a tuned guard's lost transfer — at inference, not retraining.	Base+adapter average recovers transfer and beats an equal-cost SFT+SFT ensemble (Table 8), so the gain is the base's, not generic ensembling.	Compose to *repair* a guard you already tuned; then recalibrate the threshold on the target regime (ranking recovery ≠ calibration transfer).
5. A domain change can hide the violation entirely; general-safety score ≠ compliance.	Rankings and margins shift on the mortgage domain split (Act III, Table 11; directional).	In a regulated domain, build domain-grounded, dual-labeled evaluation and a protected-pair invariance check before trusting any guard.
6. A small, single-token guard is fast, cheap, and keeps data in-house.	One forward pass ≈10–50 ms (P50–P90, batched) on one A100 (Table 15); no third-party egress.	Prefer a small self-hosted guard for inline, high-volume, or regulated traffic over a hosted-API round-trip.
7. Fine-tuning a *released* guard specializes it too; KL-SFT keeps transfer but at a represented cost.	Confirmatory 10-checkpoint study: SFT raises represented AP +0.174 (LCB +0.129) with a bound-confirmed transfer loss; KL-SFT preserves transfer (LCB +0.035) but its represented cost (LCB −0.060) fails the −0.02 non-inferiority margin (Section 4).	Adapting a purpose-built guard is not exempt from the tradeoff; use KL-SFT as a tradeoff dial, not a free upgrade, and re-measure both splits on your own data.

8 The practitioner's decision guide

1. **Always compare to your own base**, on represented *and* held-out sets, plus a benign over-refusal and a hard-attack stress set — a leaderboard delta vs. other models hides specialization.
2. **Treat composition as a repair for a guard you already tuned, not a free win over the base.** If you have SFT'd and transfer regressed, an output-space base+adapter average recovers much of what SFT gave up — and it beats an equal-cost SFT+SFT ensemble (Table 8), so the gain is the base's doing, not generic ensembling — but re-calibrate the threshold on the target regime (ranking recovery $\neq$ calibration transfer). If you have *not* tuned yet and transfer is the priority, the untuned base can already be the best transfer scorer; compose only once tuning has actually cost you transfer.
3. **In a regulated domain, build domain-grounded, dual-labeled evaluation** and a protected-pair invariance check; a general-safety score does not cover compliance.
4. **Prefer a small, self-hosted guard for inline, high-volume, or regulated traffic**: one local forward pass is ≈10–50 ms (P50–P90, batched; P99 up to ≈94 ms) (Table 15) and keeps regulated prompts in-house, versus a per-call fee and a network round-trip to a third party (Table 16).

Each step carries its caveat: these are estimates on a fixed panel, and the domain labels are a measuring stick, not a verdict.

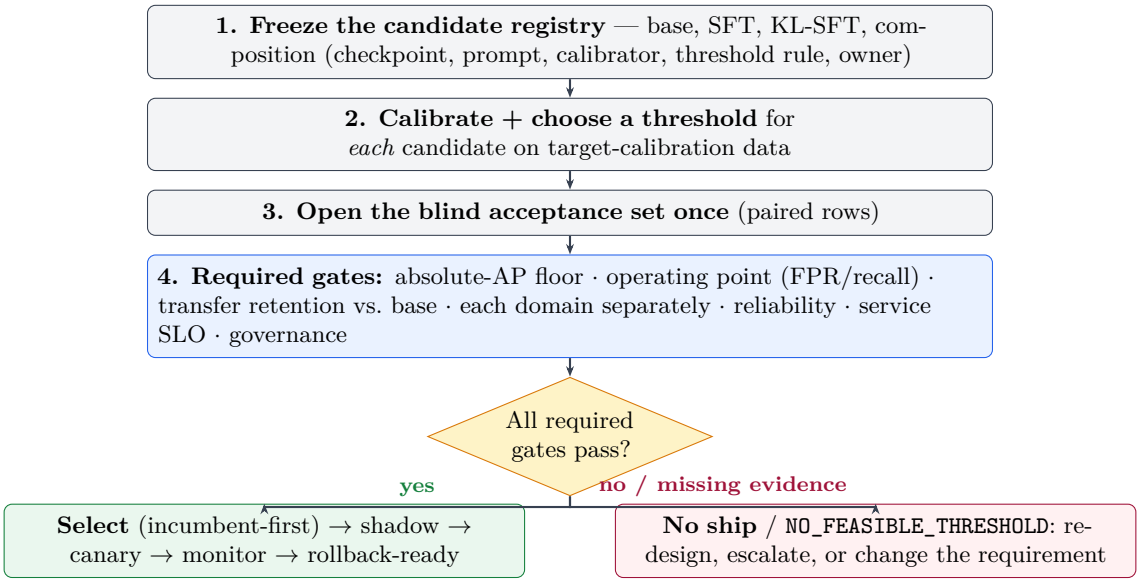

Figure 10: **Gate candidates, not leaderboards** (recommended workflow; not validated end to end by this study). Every candidate is calibrated and thresholded separately, evaluated once on a blind acceptance set, and must clear *all* required gates; a missing required gate is a failure, and an empty feasible set is a deliberate NO-SHIP, not a relaxed cutoff.

8.1 Latency and cost: the case for a small, self-hosted guard

An inline guard runs on *every* request, so its own latency and cost sit on the critical path. Because our guard emits a single verdict token — one forward pass, no autoregressive generation — it is fast to run: measured per-call latency is ≈10–50 ms (P50–P90, batched on one A100; P99 up to

≈94 ms, and composition adds a pass) depending on model size (Table 15), from 10.4 ms (P50) for Qwen2.5-1.5B to 25.2 ms for Qwen3-4B, with P90 within ≈50 ms, on a single A100 at batch 16. Latency tracks model size and prompt length, not any decode budget, because there is nothing to decode. One caveat on reading these numbers: they are *batched* per-row times (batch 16 on one A100) — throughput-latency under load, not a single-request, batch-1 serving path, which carries a higher fixed per-call overhead but no queueing. Treat them as an order-of-magnitude serving estimate on this hardware, not a single-request SLA.

Table 15: Guard inference latency — one forward pass to the single-token verdict (no autoregressive generation), per-row at batch size 16 on NVIDIA A100-SXM4-40GB (bf16), over the 79,392 committed Act I/II score rows. These are batched per-row times (throughput-latency under load), not single-request batch-1 serving latency, and composition (Act II) needs two passes. Latency scales with model size and prompt length, not with any decode budget.

Guard	P50 (ms)	P90 (ms)	P99 (ms)
Qwen2.5-1.5B	10.4	21.3	41.7
SmolLM2-1.7B	11.9	24.4	44.8
SmolLM3-3B	20.1	38.5	70.7
Qwen3-4B	25.2	48.2	93.9
All four	14.4	38.2	93.8

A frontier hosted-API guard is structurally disadvantaged on the two axes that decide an inline deployment (Table 16) — illustratively, since we benchmark no hosted API here and the frontier column is an order-of-magnitude sketch from public characteristics, not a measurement. It adds a network round-trip — typically hundreds of milliseconds — to *every* request; it bills a per-token fee that scales with traffic; and, decisively for a regulated domain, it sends every prompt to a third party. A compact guard that a team hosts, versions, and audits in-house keeps regulated data inside the boundary and adds only tens of milliseconds. Together with the specialization (Act I) and composition (Act II) findings, this is the practical case for building guards from small checkpoints rather than routing every request to a frontier model.

Table 16: Deployment economics of an *inline* guard: a small self-hosted guard versus a frontier hosted-API guard. The small-guard latency is *measured* in this study (Table 15); the frontier column is an *illustrative, order-of-magnitude* comparison from public hosted-API characteristics (typical round-trip latency and per-token list pricing as of mid-2026), *not* a measurement made here.

	Small self-hosted guard (this report)	Frontier hosted-API guard
Per-call latency	one local forward pass, ≈10–50 ms (Table 15)	network round-trip to a hosted API, typically 10^2–10^3 ms
Marginal cost / call	only amortized local compute — a 1.5–4B model serves on a commodity GPU (or CPU)	a per-token fee on every request, at the vendor's list price
Data residency	regulated prompts never leave your boundary	every request is sent to a third party — a data-governance / compliance concern
Operational coupling	self-contained; pinned, versioned, auditable in-house	external availability, rate limits, and silent model updates

9 Reproducibility

Every number above is `\input` from a committed generated table bound to a lock — none is hand-typed. A single entry point, `make reproduce` (run in `papers/unified-report/`, which calls `reproduce.py`; distinct from the repo-root `make repro` that re-verifies the release cache), regenerates all tables/figures from committed per-row scores by calling each study's analysis (the Act I specialization tables in a lock-pinned environment; the Act II composition, mortgage, and ExpGuard tables from committed scores in any environment), and asserts byte-identity with the committed report tables.

Three reproducibility tiers, kept distinct. **(i) Analysis reproducibility** — regenerating every table and figure from the committed per-row scores — is what `make reproduce` verifies byte-for-byte, with no GPU and no network, and is the tier every number in this report meets. **(ii) Training/scoring reproducibility** — re-deriving those per-row scores by training the 4×5 adapters and rescoring — requires a GPU and pinned model/data access (only re-scoring the gated ExpGuard set additionally needs dataset access). **(iii) Artifact-generation reproducibility** — regenerating the mortgage benchmark itself — is *not* claimed: its LLM construction stages run at nonzero temperature and the set is intentionally frozen, so only its *evaluation* reproduces, not its generation. Raw third-party rows are referenced by pinned identifier + revision + content hash, not redistributed.

Code and data availability. All code, data manifests, generated tables, and the frozen benchmark are public at `https://github.com/rrahimi-uci/guard-ranking-fragility`. The one-command pipeline is `papers/unified-report/reproduce.py` (`make reproduce`); the frozen, dual-labeled mortgage benchmark ships at `mortgage-benchmark/benchmark/v1_hmda2022/` (994 rows, SHA-256-checksummed, with a text-free index); the committed per-row guard scores that regenerate every number live under `artifacts/` (the Act I/II scores in `artifacts/paper_a_sft_v2/`, the mortgage baseline in `mortgage-benchmark/out_eval/`, and the finance/health/law scores in `artifacts/expguard_external/`); and the figures are built by `papers/unified-report/figures/make_figures.py`.

10 Conclusion

A small guard's benchmark score is not a fixed property of the guard — the benchmark co-produces it. Measured honestly, fine-tuning specializes rather than uniformly improves; a retraining-free composition recovers some of the lost transfer; and in regulated domains a general-safety score does not cover compliance. The disciplined next step is not a more confident headline but a prospectively locked evaluation on genuinely uninspected data.

A Related work

This report sits at the intersection of five literatures: the design of small LLM-based *guard classifiers*; the growing evidence that *fine-tuning degrades* or narrows a model's safety behavior; the study of *calibration and operating points* for moderation; *model composition* in weight versus output space; and the emerging work on *domain-specialized guarding* in regulated verticals. We review each in turn and, for each, name precisely what it establishes and what our paired, same-checkpoint, composition-aware, four-domain design adds. Our contribution is not a new model, metric, or training algorithm — it is a measurement discipline and four evaluation instruments applied to one fixed panel, so the contrast throughout is methodological rather than a claim of superior scores.

How to read this map (for the non-expert)

Almost all of the "guard" papers below ask *how good is model X?* and answer with a single benchmark number, usually next to a table of *other* models. That is a *leaderboard* question. This report asks a different, paired question: *what did the fine-tune change relative to the same model before tuning, and does that change survive on data the guard never saw?* Keep that distinction in mind — most gaps we point to are gaps between a leaderboard number and a paired, regime-split delta, not disagreements about which model is "best."

A.1 Small LLM guard classifiers

The dominant deployment pattern is a compact decoder LLM turned into a binary or taxonomy-tagged moderation head. Llama Guard [22] introduced the input–output safeguard framing and a safety taxonomy, and later iterations pushed toward smaller, cheaper, and multimodal variants [14, 33–35, 37]. ShieldGemma [47] and Granite Guardian [40] extend the recipe to other base families with broad harm taxonomies; WildGuard [19] adds one-stop coverage of prompt harm, response harm, and refusal detection; and Qwen3Guard [41] is a recent open guard on the same Qwen family two of our checkpoints come from. Beyond the general-purpose guards, several works target narrower slices or richer inference: dedicated prompt-injection detectors [36, 38, 39], reasoning- and logic-augmented guardrails [25], ensemble-of-experts moderation [15], RL-driven multilingual guardrails [9], and CPU-class or multi-stage pipelines aimed at cost [31]. The parameter-efficient adaptation we use — LoRA [21] applied to a chat model to produce a guard — is exactly the LoRA-Guard recipe [12], and standardized suites such as GuardBench [3] have made cross-model comparison routine.

What this family reports, almost without exception, is *absolute* moderation performance of *one* guard against *different* models on a benchmark. That is precisely the quantity we argue is co-produced by the benchmark and therefore uninformative about a specific fine-tune. None of these works reports the *paired* base→guard change on identical rows with identical scoring, none separates a source *represented in training* from a dataset-held-out *transfer* regime as a first-class split (our Table 1), and none treats a *retraining-free composition* as a measurable design axis. We reuse the LoRA-Guard recipe not to beat these guards on a leaderboard — we score four small open checkpoints, not the gated production guards — but to isolate what the fine-tune itself does. GuardBench and its peers are the backdrop against which our thesis ("the benchmark chooses the winner") is stated: they normalize the very comparison we show is fragile.

A.2 Fine-tuning degradation and policy / benchmark transfer

The closest prior evidence is that fine-tuning can *weaken* rather than strengthen safety. Hsiung et al. [20] show that safety guardrails can collapse after fine-tuning and attribute the collapse

to similarity between the alignment and fine-tuning data — a mechanism that rhymes with our specialization finding, but is measured on a model's own alignment behavior rather than on a guard classifier's represented-versus-transfer ranking split. Liu et al. [30] document that guardrails overfit their training *policy* and propose augmented policy training as a remedy; Li et al. [27] give a complementary mechanistic account, showing that prompt-attack defenses learn *surface heuristics* that do not generalize — essentially a why for the transfer loss we observe empirically. Bassani and Sanchez [4] probe the same fragility from the perturbation side, measuring guardrail robustness to input mutations and adversarial attacks, and Hackett et al. [18] demonstrate concrete evasion of injection/jailbreak detectors. Framed most generally, Akinrele and Gowda [1] argue that prompt-injection detection is *regime-dependent* — performance is a function of the deployment regime, not a fixed model property — which is our thesis stated for one task with interpretable structural signals.

Our addition is the measurement design rather than the qualitative claim. Where these works compare a model before and after, or one policy against another, we hold the *checkpoint, manifest, seeds, and scorer fixed* and read the paired change as a distribution over a purposively chosen panel with a hierarchical bootstrap, and we decompose it into a large represented-source gain versus a near-flat but *heterogeneous* transfer change (Table 1, Figure 3). Crucially, we do not stop at "degradation": we quantify the specialization geometry per checkpoint and per benchmark, carry it to a deployable operating point (Table 3), and then ask whether a composition (Section 5) changes it. And unlike Liu et al. [30], whose remedy retrains with augmented policies, our candidate remedy retrains nothing.

A.3 Calibration and operating points

Ranking quality (average precision) and thresholded decision quality are distinct, and the gap between them is a calibration problem. Guo et al. [17] established that modern neural networks are systematically miscalibrated and that simple post-hoc scaling helps; Liu et al. [29] specialized this to LLM-based guard models, showing they are poorly calibrated for reliable content moderation; and FlexGuard [10] moves past a single fixed threshold toward continuous, strictness-adaptive risk scoring. This literature motivates two choices we make: we fit calibrators only on a development split before reading any threshold, and we report operating-point behavior (macro- and pooled-FPR, TPR, single-class recall) *separately* from ranking.

Our specific contribution to this thread is a clean separation of the two failure modes on the *same* guards. In Act II we show that a composition can recover transfer *ranking* while its realized false-alarm rate at a fixed FPR target still misses (Table 9): recovering rank does not deliver a transferable threshold. In Act III the same lesson recurs from the other direction — for tightly clustered guard scores the fixed-threshold operating point is knife-edge and unstable across software versions, which is why we deliberately do not tabulate it there. Where the calibration literature asks "is this score a probability?", we use that machinery to make the sharper point that *ranking recovery* $\neq$ *calibration transfer*, a distinction a benchmark AP number alone hides.

A.4 Weight-space versus output-space composition

Combining models to improve robustness under distribution shift is well studied in *weight* space: WiSE-FT interpolates a fine-tuned model with its zero-shot initialization [46], and model soups average the weights of several fine-tunes [45], both trading a little in-distribution accuracy for out-of-distribution robustness at no extra inference cost. The theoretical companion most relevant to us is Kumar et al. [26], who show that *calibrated ensembles* — combining models in *output* space after

calibration — can mitigate the accuracy–robustness tradeoff under shift; this is the justification behind our composition rule.

Act II (Equation (5)) is deliberately the output-space cousin of these methods: we average the base's and the adapter's *calibrated scores* with fixed equal weights, retraining nothing. The tradeoff versus WiSE-FT/soups is explicit — output-space composition needs only comparable scores, not interpolable weights, so it is portable across checkpoints that could never be weight-averaged, but it pays for two inference passes. We position composition as a *transfer-recovery remedy for guard specialization* specifically (represented cost small, transfer recovered relative to SFT and landing above base; Tables 6 and 7), and we are explicit about what the pilot does *not* include — an actual WiSE-FT weight-space rescoring is left as a control in the roadmap, and the logit-average variant is reported only as a non-promotable ablation. The equal-cost SFT+SFT control *is* run (Table 8): base+SFT beats SFT+SFT on every checkpoint, so the transfer recovery is attributable to keeping the base rather than to generic two-model ensembling. To our knowledge this calibrated output-space average has not been evaluated as a targeted fix for the represented/transfer split in small safety guards.

A.5 Domain-specialized guarding and regulated verticals

A recent line moves guarding from generic harm to *domain compliance*. ExpGuard [7] provides expert-annotated content moderation in specialized domains including finance, health, and law — we adopt it directly as our external, expert-labeled breadth replication (Table 13; complete four-checkpoint base result). FinGuard [11] detects financial regulatory non-compliance in LLM interactions; MortarBench [44] evaluates mortgage loan-origination *agents*; and Bowen III et al. [5] measure and mitigate racial bias in LLM mortgage *underwriting*. Adjacent security-benchmark methodology such as Gate AI [16] rounds out the evaluation-design context, and our mortgage instrument is grounded in public HMDA loan-level data [13].

Each of these captures one facet our four-domain design tries to unify while keeping the honesty stance explicit. FinGuard is single-domain and, like the general guards, reports absolute detection rather than a paired base-versus-tuned transfer delta. MortarBench evaluates whether an *agent completes* an origination task, not whether a *guard detects* a compliance violation, and it does not isolate the requests that read as safe yet violate policy. Bowen III et al. [5] study bias in the model's own underwriting *decisions*, whereas we study a guard's *detection* behavior and add a protected-class *minimal-pair* invariance gate as a fairness signal that ranking alone misses. Our mortgage benchmark's distinctive construct is the *dual label* $G \times D$ (Table 11, Figure 13), whose load-bearing G0/D1 stratum — looks safe, is a violation — is exactly the case a general-safety score cannot see; the zero-shot baselines (Table 12) show these compliance violations are only moderately ranked even by the strongest base. Two honesty boundaries separate our instruments from the certified-audit reading a reader might infer: the mortgage labels are *LLM-judge, policy-card-consistent — not SME-adjudicated*, and ExpGuard supplies the strictly stronger expert-labeled tier but as a *single-label* transfer probe, not a dual-label construct. We therefore pair one domain built in depth (mortgage, dual-label plus fairness gate) with three external verticals for breadth (finance/health/law via ExpGuard), all scored on the same fixed panel used in Acts I–II so the specialization question can be asked identically across domains.

The gap this report fills

Prior work establishes, separately, that guards can be built cheaply from small LLMs, that fine-tuning can degrade or narrow safety, that guards are miscalibrated, that weight-space averaging aids robustness, and that regulated domains need their own benchmarks. What is missing — and what this report supplies on one fixed four-checkpoint panel — is a single, reproducible, estimation-only thread that (i) measures the fine-tune as a *paired, same-checkpoint* represented-versus-transfer delta, (ii) tests a retraining-free *output-space composition* as a targeted transfer-recovery remedy, and (iii) carries the same question across *four regulated domains* with a dual-label mortgage construct and a fairness invariance gate. We add no new model or metric; we add the discipline of asking, at every scale, what the benchmark contributed to the verdict.

B The shared experimental setup

Every study in this report reuses one fixed, purposively chosen panel of four checkpoints and one frozen, decontaminated 1,200-row training manifest, scored by the identical single-token $z_{\text{unsafe}} - z_{\text{safe}}$ head (Equation (1)) and the same tie-aware macro-AP, under fail-closed provenance *locks* that bind the data, code, and software versions and refuse to run if any fingerprint mismatches. This single-panel, single-manifest discipline is what makes SFT, composition, and the domain evaluations *directly comparable* rather than stapled results. The *same* four checkpoints recur across all three acts, and **Qwen3-4B is the recurring character**: the strongest base, and — as the acts will show — the one that specializes most, the one composition helps least, and the numerically highest-ranking, most protected-token-invariant zero-shot mortgage guard (within overlapping CIs — Table 12 does not license a ranking).

B.1 The fixed four-checkpoint panel and pinned revisions

The panel spans two model lineages and 1.5–4B parameters: Qwen2.5-1.5B-Instruct, SmolLM2-1.7B-Instruct, SmolLM3-3B [2], and Qwen3-4B. Each checkpoint is pinned to a fixed upstream revision, and for each we verify that `safe` and `unsafe` map to *distinct single tokens* under a fixed convention (leading-space `" safe"`/`" unsafe"` first, no-leading-space fallback), recording the token IDs and strings. Table 17 lists the panel and the frozen recipe together. The estimand is this exact four-checkpoint panel; results are not extrapolated to other scales, architectures, or vendors.

Table 17: The fixed model panel and the frozen clean-v2 LoRA-SFT recipe (shared by Acts I–II). Revisions are pinned; decision tokens are verified as distinct single tokens per checkpoint.

Checkpoint	Params	Revision (pinned, abbreviated)	Decision tokens
`Qwen/Qwen2.5-1.5B-Instruct`	1.5B	`989aa79…71aa306`	{ safe, unsafe } single-token
`HuggingFaceTB/SmolLM2-1.7B-Instruct`	1.7B	`31b70e2…46d38674`	{ safe, unsafe } single-token
`HuggingFaceTB/SmolLM3-3B`	3B	`a07cc9a…335b0ac1`	{ safe, unsafe } single-token
`Qwen/Qwen3-4B`	4B	`1cfa9a7…03b3df60c`	{ safe, unsafe } single-token

Recipe (identical for all four bases): completion-only SFT loss on the verdict + appended EOS; LoRA r=32, α=64, dropout 0.05 on {q,k,v,o,gate,up,down}; 300 optimizer steps; lr 2×10^{-4} cosine schedule, warmup 0.03; effective batch 4; max sequence length 1,024; training seeds 42–46 (five per checkpoint), shared data-order seed 42.

B.2 The single-token guard formulation and prompt rendering

Every base and every adapter is scored by the same guard formulation from Section 2.1: the prompt is wrapped in one versioned instruction template asking for a one-word verdict, and the last position's `safe`/`unsafe` logits are read and stored as $s(x)$ (Equation (1)), with the softmax probability (Equation (2)) derived from them. The *semantic* system instruction is shared verbatim across all four checkpoints, but each model's own chat template renders it differently, producing three distinct rendered-template fingerprints across the panel; each is recorded. Long user content is budgeted and truncated *before* final rendering, and the runner then asserts that the complete classification instruction and wrapper survive the truncation — a contract check motivated by an earlier artifact that could left-truncate the instruction itself. Truncation strategy and scored token counts are stored per row.

B.3 The frozen LoRA-SFT recipe

The recipe (lower panel of Table 17) is fixed identically across all four bases so that the only thing varying within Act I is the checkpoint. It uses a *completion-only* loss — the cross-entropy is applied only to the verdict token and an appended end-of-sequence marker, not to the prompt — with a LoRA adapter of rank r=32 and scaling α=64 (dropout 0.05) inserted into the attention and MLP projections (q, k, v, o, gate, up, down). Training runs 300 optimizer steps at learning rate 2×10^{-4} on a cosine schedule with 0.03 warmup and an effective batch size of 4, at maximum sequence length 1,024. At effective batch 4, the 1,200-row manifest yields exactly 300 updates — one complete exposure of the data. Each base is adapted with five training seeds (42–46) that share one data-order seed (42), giving 20 adapters; the four untuned bases are scored once and reused, for 24 model bundles in all.

B.4 The 1,200-row training manifest and exclusions

Training reads one frozen manifest and never resamples a live dataset mid-run. It draws 400 rows each from three represented sources — ToxicChat [28], Prompt-Injections, and Jailbreak-Classification — with 200 safe and 200 unsafe rows per source, selected by a hashed rank salted with the data seed and frozen as one shared row order across every checkpoint and seed (Table 18). Two datasets are *deliberately excluded from training*: BeaverTails [23], because its safety annotation labels the prompt–response interaction rather than the prompt alone, and OR-Bench [8], which is reserved for the benign stress set; including either would confound the prompt-only, dataset-held-out design. The study uses the non-commercial academic data branch (ToxicChat is CC BY-NC 4.0), so any released adapter inherits a non-commercial mark, and no third-party raw text is redistributed — only pinned identifiers, revisions, and content hashes.

Table 18: The frozen 1,200-row training manifest (Acts I–II). All training rows come from three represented sources; each contributes 400 rows balanced 200:200. BeaverTails and OR-Bench are excluded from training by design.

Represented source	Native label origin	Train rows (safe : unsafe)
ToxicChat (`lmsys/toxic-chat`)	prompt toxicity	400 (200:200)
Prompt-Injections (`deepset/prompt-injections`)	prompt injection	400 (200:200)
Jailbreak-Classification (`jackhhao/…`)	prompt jailbreak	400 (200:200)
Total		**1,200** (600:600)

B.5 Three evaluation regimes and decontamination

Evaluation is partitioned into three regimes, forming a spectrum of how "new" each test is to the guard (Figure 11 shows the full construction, from sources through the family-isolation gate to these regimes; Table 19 is the complete roster of every dataset used in this report, with its role, size, and reference).

- **Represented-source** = {ToxicChat, Prompt-Injections, Jailbreak-Classification} rows *held back* from training. These measure discrimination on sources the guard trained on.
- **Dataset-held-out transfer** = {JailbreakBench [6], XSTest [43], WildGuardTest [19], WildJailbreak [24]}, whose rows were withheld from SFT. Because these encode heterogeneous native constructs (harm, refusal, jailbreak, contrast), their macro-average mixes distinct policies; we study *dataset-held-out benchmark transfer*, not one fixed universal policy.

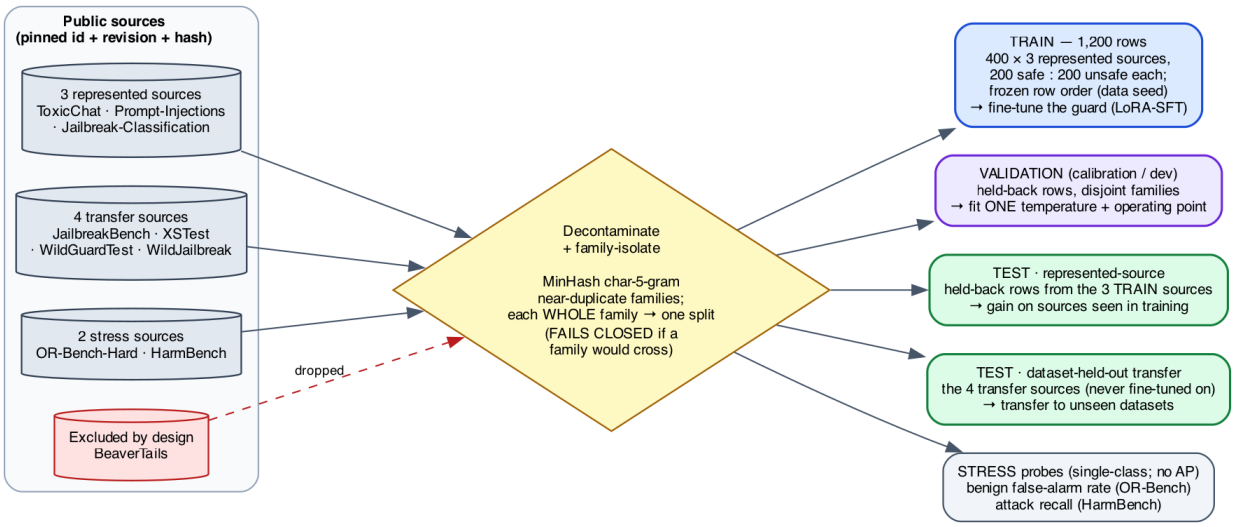

Figure 11: **How the training, validation, and test sets are built.** Public source datasets (left), each pinned by identifier, revision, and content hash, pass through a decontamination and family-isolation gate — MinHash char-5-gram near-duplicate *families* are each assigned *whole* to a single split — yielding five disjoint sets: the 1,200-row **train** manifest (fine-tuning), a held-back **validation**/calibration split (temperature + operating point only), the **represented-source** and **dataset-held-out transfer** test sets, and single-class **stress** probes. No family crosses the train/test or validation/test boundary, and the build *fails closed* if one would. BeaverTails is excluded entirely (it labels the prompt–response interaction, not the prompt).

- **Stress** = {OR-Bench-Hard benign portion (false-positive rate only), HarmBench [32] (recall only)}. These are single-class probes: **no AP or AUROC is computed on stress data**; they only watch for over-blocking (benign FPR) and missed attacks (recall).

The primary metric within each regime is benchmark-macro AP; the secondary metric is the calibration-targeted operating point of Appendix B.6.

Decontamination (family-isolated splits, MinHash). To keep near-duplicate rows from straddling the train/evaluation and calibration/test boundaries, the builder first preserves the authoritative upstream conversation, pair, and scenario identifiers, then adds deterministic character 5-gram *MinHash* edges between near-duplicate rows. It forms the connected components of that graph — "families" — and assigns *whole families* to a single split, so that no family can appear in both training and any reported evaluation, or in both calibration and any reported test or stress surface. The build *fails closed* if any family crosses a forbidden boundary. This family graph also supplies the Poisson(1) re-count weights used by the bootstrap (Section 2.6). An independent audit of 24 hard assertions covers the source exclusions, schema and row identity, exact and conflicting-label overlap, pinned revisions and content hashes, selection provenance, family disjointness, licenses, and near-duplicate dispositions. This is a *build-time* gate (family isolation plus the 24 assertions), and it is distinct from the *formal retrospective overlap audit* of the manifest against the current v2 transfer suite, which remains *pending* (Appendix E.1): the gate guarantees no near-duplicate family straddles a split within the constructed graph, but a from-scratch n-gram / near-duplicate check of the 1,200-row manifest against the v2 transfer benchmarks has not yet been run — and any residual overlap it found would make specialization look *milder*, so this is a

Table 19: Every dataset and benchmark used in this report, grouped by role. The three represented sources supply both the fine-tuning manifest (400 rows each, 200:200 safe:unsafe) and, on held-back rows, the represented-source test; the transfer and stress sets are never seen in fine-tuning. Act III adds two regulated-domain benchmarks. All sources are consumed by pinned identifier, revision, and content hash; "rows" are the counts *used here*, not necessarily the full upstream release.

Dataset (source)	Native task / label	Rows used	Ref.
Represented sources — fine-tuning manifest + held-back represented test			
ToxicChat (`lmsys/toxic-chat`)	prompt toxicity	400 train	[28]
Prompt-Injections (`deepset/prompt-injections`)	prompt injection	400 train	—
Jailbreak-Classification (`jackhhao/…`)	prompt jailbreak	400 train	—
Dataset-held-out transfer test — never fine-tuned on; benchmark-macro AP			
JailbreakBench	jailbreak robustness	held-out	[6]
XSTest	exaggerated-safety / refusal	held-out	[43]
WildGuardTest	prompt harm / refusal	held-out	[19]
WildJailbreak	in-the-wild jailbreak	held-out	[24]
Stress probes — single-class; no AP/AUROC			
OR-Bench-Hard (benign)	over-refusal (benign FPR)	benign only	[8]
HarmBench	red-team attacks (recall)	attacks only	[32]
Excluded from training by design			
BeaverTails (`PKU-Alignment/BeaverTails`)	prompt+response safety	—	[23]
Regulated-domain benchmarks (Act III)			
Mortgage `v1_hmda2022` (ours)	dual $G\times D$ + protected pairs	994	[13]
ExpGuard (`6rightjade/expguardmix`)	finance / health / law prompt safety	2,275	[7]

caveat in the direction of caution.

B.6 Estimands and the statistical protocol

For regime R, checkpoint b, and seed r, the per-regime metric is the equal-weight macro-AP over that regime's benchmarks,

$$M_R(b, r) \;=\; \frac{1}{|K_R|} \sum_{k \in K_R} \mathrm{AP}_k(b, r), \tag{8}$$

using the tie-aware, non-interpolated `scikit-learn` AP. The paired base-to-tuned change for one cell is

$$\Delta_R(b, r) \;=\; M_R(\mathrm{SFT}_{b,r}) \;-\; M_R(\mathrm{base}_b), \tag{9}$$

and the fixed-panel aggregate averages the four per-checkpoint deltas *without* resampling checkpoint identities,

$$\bar{\Delta}_R \;=\; \frac{1}{4} \sum_{b} \Big[\tfrac{1}{5} \textstyle\sum_r M_R(\mathrm{SFT}_{b,r}) \;-\; M_R(\mathrm{base}_b) \Big]. \tag{10}$$

The headline object is the joint vector $\theta = (\bar{\Delta}_{\text{represented}}, \bar{\Delta}_{\text{transfer}})$, kept as a two-dimensional quantity and read off the *specialization plane* of Figure 3 — horizontal axis the represented change, vertical the transfer change, four quadrants (uniform gain, specialize, uniform loss, transfer-favored) — rather than collapsed into a single "specialization score." The axes and quadrant interpretation are result-independent.

Uncertainty. We attach 95% two-sided intervals with 10,000 paired hierarchical-bootstrap replicates (fixed RNG seed 20260712): checkpoint identities are held fixed, SFT seed indices are resampled *within* each checkpoint, and one Poisson(1) weight is drawn per recorded family (Appendix B.5). Leave-one-checkpoint-out and leave-one-transfer-benchmark-out values are *deterministic* sensitivity checks, not resampling or population inference. The locked analysis mode is *precision-focused*: we report estimates, intervals, and sensitivities, with no intersection–union test, multiplicity correction, bootstrap p-value, or pass/fail gate.

Calibration-only operating point. As a secondary deployment diagnostic, we fit one positive temperature per bundle on the calibration split, then choose the threshold that maximizes calibration recall subject to a one-sided 95% Clopper–Pearson *upper* bound of at most 5% on pooled calibration-negative FPR; if no cutoff qualifies, the cell reports `NO_FEASIBLE_THRESHOLD` as an outcome rather than relaxing the target. Temperature and threshold are fit *only* on calibration rows — never on transfer or stress data — and the audit hard-asserts that calibration shares no family with any reported test or stress surface. The Clopper–Pearson bound assumes independent Bernoulli negatives, so it is a pooled diagnostic, not a family-aware production guarantee. The resulting operating-point rates appear in Table 3 (Act I) and Table 9 (Act II).

B.7 The composition protocol

Act II's remedy needs no retraining. For a single input we run the base and one SFT adapter, convert each raw score to a probability with a calibrator fit only on a development split, and report the fixed equal-weight average defined later as Equation (5). The $\frac{1}{2}$ weights are fixed in advance (never tuned on test rows), so the only cost is the second inference pass. This is the output-space composition of Section 2.8; the logit-average and convex-weight variants were visible during development and are reported only as non-promotable ablations.

B.8 Domain evaluation instruments: mortgage and ExpGuard

Act III evaluates the *same four checkpoints* on two complementary regulated-domain instruments.

Mortgage (built in depth, dual-label). We constructed a fixed, HMDA-grounded benchmark whose unit is one incoming request carrying two independent labels, general-safety G and mortgage-policy D (Section 2.9). Requests are grounded in de-identified, banded HMDA fact sheets — never verbatim records — through an agentic pipeline that plans, grounds, generates, adversarially mutates, and rubric-bound-judges each item, accepting it only if its assigned label matches the target, before a provenance-tracked, family-isolated split into the frozen release (Figure 13). The benchmark has 994 rows over four $G \times D$ quadrants — G0/D0 (450), G0/D1 (502), G1/D1 (42), and G1/D0 (0, empty by construction) — with the load-bearing **G0/D1** stratum (reads safe, is a violation) the largest. It spans seven content strata (fair-lending 204, fraud 112, UDAAP 90, disclosure 66, ATR/QM 54, privacy 18, and benign 450) and is split into train 604 / dev 149 / public-test 146 / private-test 95. As a fairness-invariance gate it embeds 39 protected-class *minimal pairs* (78 rows) that are identical except for one protected token; the induced prediction gap is denoted Δ_{context}. The row composition is in Table 11 and the zero-shot base results (AP·G, AP·D, Δ_{context}) in Table 12 (Act III).

A measuring stick, not a legal finding

Mortgage labels are assigned by an *LLM judge* against written policy cards — *not* SME-adjudicated (self-consistency only; no human Fleiss-κ). The G1/D0 quadrant is empty and some strata are small. The benchmark *surfaces* guard behavior; it certifies nothing about any real lender or model, and licenses no fair-lending claim.

Finance, health, law (external breadth: ExpGuard). To test whether the specialization/-transfer pattern recurs across regulated verticals *with expert labels* — a strictly stronger labeling tier than the mortgage LLM-judge — we also score the same four checkpoints on ExpGuard [7], an expert-annotated moderation set of 2,275 rows spanning **finance**, **health**, and **law**, using its `prompt_label` for input-prompt classification (matching our task). This is a single-label transfer probe, not the mortgage dual-$G \times D$ construct — one domain built in depth plus three external verticals for breadth. We report aggregate and per-domain AP for the four base checkpoints (Table 13, Figure 9). The four-checkpoint result is complete and reproducible from committed text-free per-row scores; the tuned comparison is future work.

C Benchmark construction and mechanism detail

C.1 The attractor: post-SFT scores are benchmark-fixed, so "stronger bases specialize more" is arithmetic

The cleanest way to see what Act I does and does not show is to notice a regularity that runs underneath Table 1: **fine-tuning behaves like an attractor**. No matter where a checkpoint starts, SFT pulls its ranking to nearly the *same* score on a given benchmark. Represented-source AP after SFT is 0.982 ± 0.005 across the four checkpoints; transfer AP after SFT is 0.807 ± 0.024 — both far tighter than the *base* spread they came from (0.178 and 0.073 respectively; the transfer variance shrinks about **9-fold**). Figure 12 (left) shows the collapse: four widely spread base scores, one narrow post-SFT band.

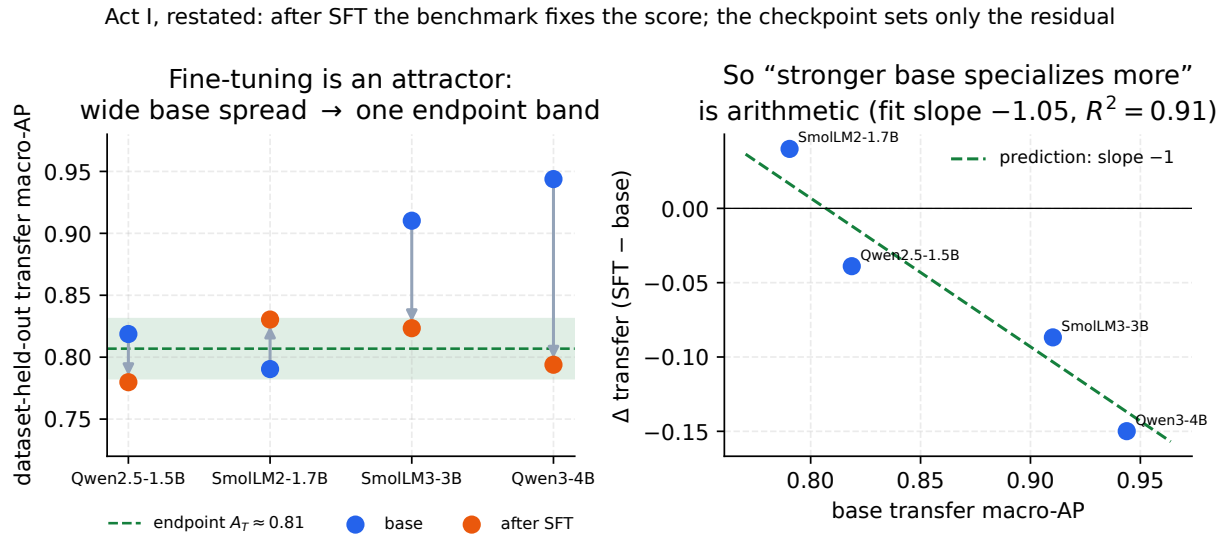

Figure 12: **The fine-tuning attractor.** *Left:* on the transfer benchmarks the four base checkpoints span 0.79–0.94 macro-AP, but after SFT they collapse into a narrow band around a benchmark-fixed endpoint $A_T \approx 0.81$. *Right:* because every checkpoint lands near that same endpoint, the paired change Δ = SFT − base is forced onto a line of slope -1 (green); the four points sit on it (fitted slope -1.05, $R^2 = 0.91$). "Stronger bases specialize more" is therefore arithmetic, not a discovered behavioral law.

Why this reframes "stronger bases specialize more." Once the endpoint is (approximately) fixed, the change we plot is pure subtraction: $\Delta = \text{endpoint} - \text{base}$. A stronger base then *must* show a smaller Δ — not because strong models are intrinsically fragile, but because they start closer to the ceiling. This is testable. Regressing Δ on the base score across the four checkpoints gives slope -0.99 (represented, $R^2 = 0.999$) and -1.05 (transfer, $R^2 = 0.91$) — the -1 the attractor *predicts before any fitting*. One caveat on reading those numbers: the represented $R^2 = 0.999$ is close to *definitional*. When the endpoint band is as tight as it is on represented sources (± 0.005), Δ = endpoint − base is arithmetically almost a slope--1 line in the base score, so a near-perfect fit is nearly forced and is not independent confirmation of anything. The transfer fit, where the endpoint band is several times wider, is the load-bearing test — and it still lands at slope -1.05. Either way the eye-catching ordering (SmolLM2 $+0.040$ down to Qwen3-4B -0.150) carries no behavioral content beyond the endpoints themselves.

Where the real content lives. Two places, both benchmark-owned rather than model-owned. First, *where the endpoints sit*: $A_T \approx 0.81$ is largely a property of the *(training manifest, benchmark)* pair — to the precision of the small residuals below, the base model has largely dropped out of the equation, and the post-SFT score is *largely determined by* the manifest and the benchmark rather than by which checkpoint you started from. This is the strongest form of the paper's thesis: after SFT, the benchmark and the manifest largely choose the score. Second, the *small residuals* around the endpoint (at most 0.027 on transfer): these measure how much of a base's identity survives the fine-tune, and are arguably a more honest per-checkpoint quantity than Δ itself. LoRA only adds a low-rank correction in the base's own features, so full base-independence is approximate by construction — the residual is exactly that approximation, made visible.

Why the represented ceiling is ≈ 0.98, not 1.0 — and why that helps the thesis

A ranker of the *latent* truth generally cannot score AP = 1 against slightly noisy *observed* labels (though it can rank the observed labels perfectly if the noise is systematic and learnable): under random label noise a small rate η of ambiguous or mislabeled rows lowers the expected achievable AP to roughly $1 - 1.3\,\eta$ (the coefficient 1.3 is an order-of-magnitude rule of thumb, not derived here — the exact factor depends on prevalence and on where mislabels fall in the ranking). The observed common ceiling 0.982 is what you would see at $\eta \approx 1.5\%$ label noise — i.e. the ceiling would then be a property of the benchmark's *labels*, not of the models. This is a *hypothesis*, not a measured result: it is checkable by auditing the top-ranked "false positives" of the SFT guards — under this reading most should be mislabels — but we have not run that audit, so the label-noise explanation of the ceiling remains conjectural. Either way, the empirical fact we rely on is only that a common ceiling exists and is shared across checkpoints; the benchmark, not the model, sets the top of the scale.

How much to trust the "base strength" ordering

The apparent trend rests on *four* checkpoints — and because seeds within a checkpoint are highly correlated (every Qwen3-4B seed is negative, every SmolLM2 seed but one positive), the effective sample is $n = 4$, not 20. But the attractor turns the checkpoint-level "specialize" pattern into a *prediction* from a single constant: a checkpoint specializes exactly when its base transfer score exceeds the endpoint ($\text{base}_T > A_T \approx 0.81$), which selects Qwen2.5, SmolLM3 and Qwen3-4B as specializers and leaves SmolLM2 in uniform-gain — the checkpoint-mean split we observe, and a 15/5 seed count. The match is at the checkpoint mean, not seed-perfect: two of the twenty seeds cross their checkpoint's boundary (Qwen2.5's seed 42 gains +0.008; SmolLM2's seed 46 loses −0.001), so the constant predicts the aggregate split rather than every individual seed. The sharper, falsifiable hypothesis to carry forward is therefore *endpoint invariance* itself: any new 1.5–4B instruct checkpoint tuned with this recipe should land near represented 0.98 and transfer 0.81 regardless of where its base started. One honest limit on this extrapolation: our four checkpoints are only *two* lineages (two Qwen, two SmolLM), so what we have actually shown is endpoint invariance *within* two families — a prospective test (roadmap item 7) should add unrelated lineages before the invariance is treated as recipe-general. What Act I establishes now is the qualitative claim, robust across all four checkpoints and 20 seeds — SFT buys represented-source ranking and not, on average, transfer.

C.2 HMDA grounding: de-identified, banded fact sheets

Realism is what makes the benchmark hard: a guard should face requests that read like genuine mortgage-workflow traffic, not toy prompts. We ground each scenario in the public HMDA 2022 loan-level snapshot [13], pulled from the FFIEC/CFPB Data Browser (the U.S. agencies that collect and publish HMDA data). Each source record is reduced to a *banded, de-identified fact sheet*: loan purpose, occupancy, banded loan amount, banded income, banded LTV and DTI, action taken, denial reason, and state — and never an exact dollar amount, a census tract, or any identifier.

Three properties make this PII-safe by construction. **(i) Banding**: every exact figure is replaced by a range (an income band rather than a dollar amount, likewise for LTV and DTI). **(ii) Marginal sampling**: each field is drawn from its own marginal distribution over the bands — how often each band occurs *on its own* — rather than from the real joint combinations of fields in any one record, so no actual borrower's row can be reassembled. **(iii) A build-time assertion** that no emitted fact sheet reproduces a single source record verbatim; the frozen release carries `contains_real_pii=false` with zero violations at freeze. Following Bowen III et al. [5], the fair-lending and ability-to-repay cells are deliberately biased toward **borderline, higher-risk** files (high DTI, high LTV, a prior denial) — exactly where underwriter discretion, and therefore bias, has room to operate.

"Banded," "marginal," and "PII-safe" — in plain words

Banded means we never print an exact number; "income $92,450" becomes "income in the $75k–$100k band." *Marginal sampling* means we build a synthetic applicant field-by-field from how common each band is by itself, not by copying a real person's combination of fields — so even though every band is realistic, the assembled applicant corresponds to no real borrower. *PII-safe* (personally identifiable information) then follows: with no exact figures, no tract, and no real field-combination, there is nothing in a fact sheet that could re-identify anyone. The grounding buys realism; the banding and marginal sampling buy privacy.

C.3 The agentic construction pipeline

Rows are authored and labeled by a five-stage agentic loop, summarized in Figure 13. The loop is *deterministic in structure* (the coverage cells it must fill are enumerated up front) but *stochastic in surface wording* (the LLM stages run at temperature > 0), which is why the release is a frozen artifact rather than a regenerable one.

1. **Planner.** Enumerates coverage cells as a product of quadrant × trap type × policy card × role × protected context, so every intended combination is targeted rather than left to chance.
2. **HMDA-grounder.** Draws a de-identified banded fact sheet (Appendix C.2) matching the planned cell.
3. **Generator.** Authors the request in a mortgage-workflow *role voice* — applicant, loan officer, underwriter, processor, broker, or adversary — so the phrasing matches who would plausibly send it.
4. **Adversarial mutator.** Composes *label-preserving* tactics that make a violation subtle and deniable without changing its ground truth: euphemism, coded proxy, business justification, and buried injection.
5. **Rubric-bound judge.** Assigns G and D by reading the request against the policy cards (Appendix C.4).

The loop closes on an **accept-iff-target** rule: a generated row is kept only when the judge's assigned label matches the target the planner set for that cell; otherwise the row is retried or dropped. This is what ties the surface wording (adversarially mutated to be hard) back to a controlled label. Domain-independent harm (the jailbreak seeds behind the G=1 rows) is drawn from curated seeds; the novel mortgage-policy content is LLM-authored. Accepted rows then pass provenance and decontamination, a family-isolated train/dev/test split with a sealed private test, and are published as the frozen `v1_hmda2022` release — the object the reproducible harness scores.

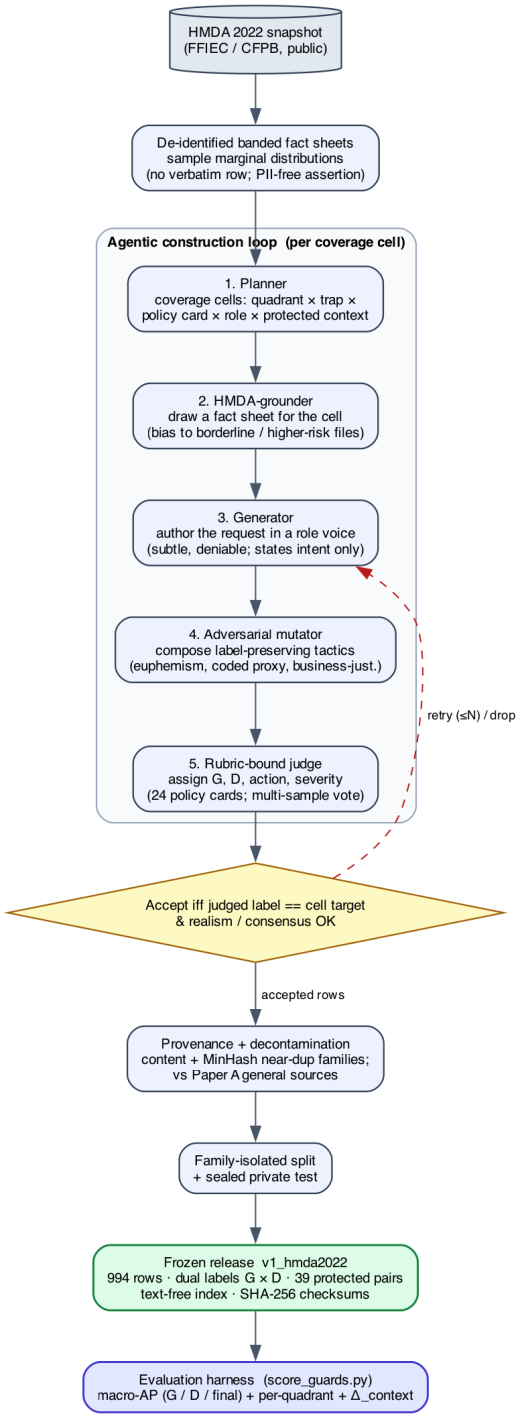

Figure 13: The agentic mortgage-benchmark construction pipeline: real HMDA records → de-identified banded fact sheets → plan / ground / generate / adversarially-mutate / rubric-bound-judge → accept-iff the judged label matches the planned target → provenance + family-isolated split → the frozen `v1_hmda2022` release scored by the evaluator.

Each row is built around exactly one **trap** — the tactic that makes its label hard to read off the surface. `direct` states the ask plainly; `business_justified` wraps it in a business rationale; `coded_proxy` substitutes a stand-in (a neighborhood, a language) for a protected group; `euphemism` softens the wording; `occupancy_temptation` invites lying about owner-occupancy; `buried_injection` hides an instruction inside otherwise ordinary text; `over_refusal_bait` looks alarming but is in fact benign, so a good guard should *not* flag it; `benign_info` is a plain safe request; and `minimal_pair` is the protected-class counterfactual of Section 6.3.

C.4 The 24 policy cards and the label rubric

The mortgage-policy label D is not a vibe — it is defined by **24 benchmark policy cards** spanning six regulatory families: fair lending, ability-to-repay / qualified-mortgage (ATR/QM), disclosures, UDAAP, fraud, and privacy. Each card is a one-page rule with two machine-usable parts: an **"intervene iff" predicate** (the precise condition under which a guard *should* step in) and an **authority pointer** (the law or regulation the predicate rests on). The judge reads a request against the applicable cards and emits $G \in \{\text{safe}, \text{unsafe}\}$ and $D \in \{\text{allow}, \text{intervene}\}$; the composed final of Equation (6) follows mechanically, together with an *action lattice* (the ordered menu of responses, from allow through warn to block) and a *severity* grade.

The honesty caveat is load-bearing and stated plainly: **these labels are policy-card-consistent, assigned by an LLM judge — they are not SME-adjudicated.** The judge is run rubric-bound against the applicable cards at temperature 0 with a three-sample majority vote; all 24 cards remain unsigned, and judge agreement is measured as *self-consistency* (re-running the same judge and checking it agrees with itself), not as a human inter-rater study. Two transparency gaps we flag rather than resolve here: the frozen artifact does not record the exact judge *model and version*, so its possible model-family overlap with an evaluated checkpoint cannot be ruled out as a source of correlated blind spots; and per-label self-consistency *counts* were not exported, so self-consistency is asserted, not tabulated.

Policy cards, "SME," and Fleiss-κ — in plain words

A *policy card* turns a slice of mortgage law into something a program can check: a plain "intervene when …" condition plus a citation to the rule it comes from. An *SME* is a subject-matter expert — here, a compliance lawyer — and *Fleiss-κ* is a standard number between 0 and 1 for how much several *independent human* raters agree beyond chance, the usual gold standard for a labeled benchmark. We do not yet have that. What we report instead is *self-consistency*: the same LLM judge, re-run, agreeing with itself — a much weaker guarantee. That gap is precisely why every result in this section is a diagnostic, not a certified fair-lending finding.

D Ensembling: when does combining guards recover transfer?

Ensembling is a classic way to make a classifier generalize, so it is natural to ask whether it repairs the transfer loss that fine-tuning induces (Section 3). We test this *retrospectively* on the adaptation panel's committed per-row margins (Section 4): 9 checkpoints across 5 families (excluding the degenerate Llama-Guard null cell), each scored under {unmodified base, SFT, KL-SFT} × five seeds on the *same* rows. Every number is equal-family macro-AP on the raw logit margin, anchored to each checkpoint's *own* unmodified base — identical estimand to the main text. Within a checkpoint the base and its adapters share one output head, so raw margins are scale-comparable and can be averaged directly; across checkpoints they are not, so the cross-model committee (below) rank-normalizes first. This appendix is retrospective on the inspected panel, not a preregistered claim.

Ensembling a fine-tune with itself is only denoising. Averaging the five SFT seeds into one guard raises transfer by a bootstrap-significant but small +0.025 (one-sided LCB +0.020) over a single SFT run — yet transfer still lands *below* the base (−0.053 vs. base; Table 20). Seed-ensembling KL-SFT behaves the same (−0.008). The reason is structural: every seed is trained by the same recipe on the same data, so all members share the *same* specialization bias. Averaging them cancels seed *variance*, not the shared *bias* toward the represented sources — so it polishes a specialized guard without un-specializing it.

Ensembling across the specialization axis recovers transfer. The one ensemble that lifts transfer above the base is the base ⊕ adapter combination — which is exactly the Act II composition (Section 5). On this panel it raises transfer by +0.014 over the base (one-sided LCB +0.008 > 0), recovering +0.066 relative to pure SFT, at a represented cost of −0.022 (95% CI [-0.035,-0.013]). It is a recovery, not a Pareto win: for the strongest base the composed guard still gives back a little transfer (the Act II caveat, Section 5). Figure 14 makes the distinction visual: the seed-ensembling arrows stay inside the "transfer below base" band, while the arrow to base ⊕ SFT crosses into "transfer recovered." Adding KL-SFT as a third member (base ⊕ SFT ⊕ KL-SFT) lands essentially on top of base ⊕ SFT — no further gain. The active ingredient is *diversity across the specialization axis*: the un-tuned base makes off-distribution errors that are decorrelated from the tuned member's, so averaging cancels them; two specialized members do not. This is measured directly, not merely inferred from the AP midpoint: on transfer rows the base's per-row errors correlate only 0.422 with its own fine-tune's, versus 0.851 between two fine-tune seeds — the base is far more complementary to the fine-tune than one fine-tune run is to another, which is exactly why base ⊕ SFT recovers transfer while SFT ⊕ SFT (below) does not.

A committee of fine-tuned guards does not generalize better. The strongest form of the technique — a cross-model *committee* that rank-averages different guards on the same row — makes the point sharpest (Table 21). A committee of the SFT guards is *worse* than the single best guard on transfer (−0.014 over the four general checkpoints, −0.056 over all ten): SFT drives every member toward the same represented sources, so their held-out errors are correlated and averaging cannot cancel them — it only dilutes the best member. The committee of *KL-SFT* guards merely breaks even (+0.004 / −0.004), because KL regularization leaves the members less specialized and therefore more diverse — but that is the KL penalty doing the work, not the ensemble.

Table 20: Retrospective ensembling on the adaptation panel (9 checkpoints across 5 families, excl. the degenerate Llama-Guard null cell; equal-family macro-AP on the raw margin, anchored to each checkpoint's *own* unmodified base). Within-checkpoint ensembles average raw margins (scale-comparable, shared head). Represented = id_test, transfer = transfer_test. Ensembling a fine-tune *with itself* (seeds) only denoises; only adding the *un-tuned base* lifts transfer above base.

	represented		transfer		
Guard / ensemble	AP	Δ	AP	Δ	effect
unmodified base	0.776	—	0.906	—	reference
Ensemble a fine-tune with itself (redundant ⇒ correlated errors)					
SFT (single run)	0.984	+0.208	0.828	−0.078	specializes
SFT, 5-seed ensemble	0.989	+0.213	0.852	−0.053	denoise; still < base
KL-SFT (single run)	0.941	+0.165	0.886	−0.019	mild specialize
KL-SFT, 5-seed ensemble	0.947	+0.171	0.897	−0.008	≈ base
Ensemble across the specialization axis (diverse ⇒ decorrelated errors)					
base ⊕ SFT (= Act II)	0.967	+0.191	0.920	+0.014	**recovers transfer**
base ⊕ KL-SFT	0.886	+0.110	0.915	+0.009	recovers, lower rep
SFT ⊕ KL-SFT	0.986	+0.210	0.895	−0.011	no transfer help
base ⊕ SFT ⊕ KL-SFT	0.968	+0.192	0.918	+0.013	≈ base⊕SFT

Does ensembling help here?

Only when it injects the diversity fine-tuning removed. Keeping the *un-tuned base* in the ensemble (Act II composition) recovers transfer above base; ensembling fine-tunes *with each other* — across seeds or as a committee of tuned guards — does not, because specialization correlates their off-distribution errors. Ensembling is a variance tool, not a bias fix: it costs N inference passes, does not dissolve the represented/transfer tradeoff, and every composed candidate must still be recalibrated and gated on both splits like any other (Section 8).

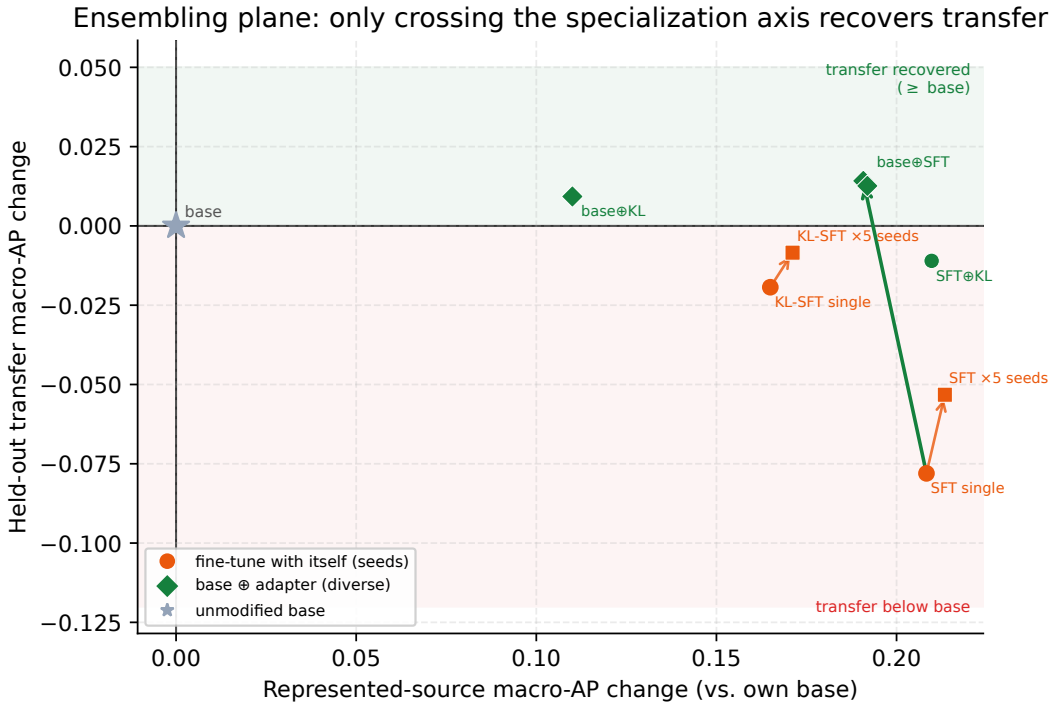

Figure 14: **The ensembling plane** (equal-family mean, 9-checkpoint panel). Change in represented (x) and held-out transfer (y) macro-AP vs. each checkpoint's own base. Ensembling a fine-tune with itself (orange, single → 5 seeds) stays in the "transfer below base" band; only crossing the specialization axis by adding the un-tuned base (green arrow, to base ⊕ SFT) lifts transfer above the base. base ⊕ SFT ⊕ KL-SFT coincides with base ⊕ SFT (adding a second specialized member buys nothing).

Table 21: Cross-model *committee* ensembles (rank-percentile average of different guards on the same row). A committee of SFT guards does not beat the single best guard on transfer — SFT correlates their off-distribution errors. Only the less-specialized KL-SFT committee breaks even.

Committee	rep. AP	transfer AP	best single (tr.)	Δ vs best
4 general checkpoints				
committee of bases	0.666	0.921	0.942	−0.022
committee of SFT guards	0.990	0.854	0.868	−0.014
committee of KL-SFT guards	0.975	0.918	0.914	+0.004
all 10 checkpoints				
committee of bases	0.833	0.970	0.970	−0.001
committee of SFT guards	0.992	0.876	0.932	−0.056
committee of KL-SFT guards	0.975	0.945	0.949	−0.004

E Limitations, the evidence ledger, and the validation roadmap

This section is the report's honest accounting. Everything above is a set of *measurements*; this section states precisely what those measurements do and do not license you to conclude, records the provenance and label tier of each body of evidence in one unified ledger, and lays out the concrete steps that would upgrade each estimate into something stronger. Two rules are enforced without exception and should be read as the spine of the whole report: **(1)** retrospective, estimation-only evidence is *never* pooled with external or prospective evidence, and **(2)** no number anywhere is promoted into a causal, universal, or fair-lending claim.

The difference between "what we measured" and "what you may conclude"

Most of this report reports *paired differences* on a *fixed* set of four models: for each checkpoint we compare the guard to its own base on identical rows with identical scoring. That design is unusually clean for what it targets — it removes the model-to-model confound that ordinary leaderboards suffer — but it buys that cleanliness by giving up breadth. A paired difference on four hand-chosen checkpoints tells you what happened *to these four models on these rows*; it does not, by itself, tell you what will happen to a fifth model, on a production traffic mix, or why. The bootstrap intervals quantify sampling noise *conditional on this panel and these benchmarks*; they are not confidence statements about a population of models or prompts, and they are not hypothesis tests. Keeping that distinction visible is the entire point of this section.

E.1 What these results do NOT establish

Cross-cutting claims we do not make. The following limits apply to *every* act, regardless of which axis is in view.

1. **No causal claim.** We observe that fine-tuning *co-occurs* with specialization (Act I, Section 3) and that composition *co-occurs* with transfer recovery (Act II, Section 5). We do not isolate a mechanism. In particular, the reading that "stronger bases specialize more" — suggested by the ordering from SmolLM2-1.7B (+0.0400 transfer change) down to Qwen3-4B (−0.1499) — is a *hypothesis drawn from four points*, not a demonstrated law. It is also mechanically entangled with the metric: because the paired change is $\Delta = \mathrm{AP}_{\mathrm{SFT}} - \mathrm{AP}_{\mathrm{base}}$ and every SFT guard converges to ≈ 0.98 represented-source AP regardless of where its base started ($0.4524 \to 0.9806$ for SmolLM2 up to $0.8855 \to 0.9837$ for Qwen3-4B), Δ is arithmetically coupled to the base level. A negative correlation between base AP and Δ is partly a definitional artifact and cannot be read as evidence of a behavioral tendency.

2. **No population or universal claim.** The panel is a *fixed, purposively chosen* set of four checkpoints drawn from only *two model lineages* (Qwen: Qwen2.5-1.5B, Qwen3-4B; and the SmolLM family: SmolLM2-1.7B, SmolLM3-3B). Lineage, scale, and native alignment recipe are therefore confounded: any "size effect" we might read off four points is inseparable from "which of two families it came from." Nothing here estimates a distribution over models, and the hierarchical-bootstrap intervals (e.g. the aggregate transfer change -0.0589 $[-0.0837, -0.0321]$) are explicitly *conditional on this panel*, not population intervals.

3. **Heterogeneous native policies are not controlled.** Each base checkpoint arrives with its *own* native safety training and implicit policy. A paired base→SFT delta therefore mixes "what our fine-tune did" with "what this vendor's alignment already did," and the four bases do not share a common definition of `unsafe`. This is a feature for the paired design (we always compare a model to itself) but a limit on interpretation: the cross-checkpoint spread partly

reflects incompatible starting policies, not just differing tunability.

4. **Not confirmatory — the benchmarks were inspected during development.** The 1,200-row training manifest and the transfer suite were visible to the researcher while the method was being built. "Held out" throughout this report means *dataset-held-out* (rows/sources not used in training), *not* sealed-from-the-researcher. This makes Acts I and II *retrospective and estimation-only*. No pre-registration protects them; the honest status is "a reproducible characterization of this panel," not "a confirmed finding." The one exception is the starting-type adaptation study (Section 4), whose estimands, decision rules, and non-inferiority margin were locked in a committed claim registry (`artifacts/starting_type_adaptation_v1/protocol/claim_registry.json`, hashed in the release lock) *before* any score existed — it is confirmatory (though still conditional on the fixed model panel).

5. **Decontamination against the v2 transfer suite is asserted, not yet audited.** The manifest was decontaminated during construction and is reported as clean-v2, but the *formal* overlap audit (n-gram / near-duplicate check of the training manifest against the current v2 transfer benchmarks) is still pending. If residual train/transfer overlap exists, it would *inflate* measured transfer — i.e. it would make specialization look milder than it is — so the pending audit is a caveat in the direction of caution.

6. **Single recipe, single manifest.** Act I uses exactly one LoRA configuration (rank 32, α=64, dropout 0.05 on `q,k,v,o,gate,up,down`; 300 steps; lr 2×10^{-4} cosine, warmup 0.03; effective batch 4; `max_len` 1024; seeds 42–46 with data order fixed at seed 42) on one frozen manifest. The specialization signature is a property of *this recipe on this data*; a different rank, step budget, or data mixture could move the represented/transfer trade-off, and we do not sweep them.

7. **Balanced-prevalence evaluation overstates production precision.** The transfer regime is scored on a balanced pool (790 unsafe vs. 790 safe rows) and the represented regime is near-balanced (313 vs. 364). Real inbound traffic is overwhelmingly benign, so unsafe prompts are *rare*. Average precision and any precision-at-threshold measured on a balanced set are optimistic relative to a deployment where the negative class dominates: at low base rates the same ranking yields far lower precision. This is now made quantitative rather than left as a caveat — Section 3.7 and Figure 4 recompute $\mathrm{AP}(\pi_+)$ exactly from the ranking (Equation (4)): the base transfer guards fall from 0.79–0.94 balanced to $\approx$ 0.10–0.54 at 1% prevalence (Figure 4), and the guard ordering itself re-spaces as positives get rare. Read every AP here as a *ranking* quality on a balanced pool, not as a deployed precision.

8. **Ranking recovery is not threshold transfer.** All AP-based results are threshold-free; deployment is not. At a calibration-selected operating point the gap is stark: SFT lifts represented-source recall from 13.0% to 76.9% but raises the transfer false-alarm rate from 8.1% to 15.5% (macro) — and pooled FPR moves further, 4.3%$\rightarrow$ 17.0% — while HarmBench recall *falls* from 78.0% to 60.0% (Table 3). Composition recovers rank but its realized transfer FPR at a 5% target is 11.4% — better than SFT's 15.5% yet still above target (Table 9). A better AP does not hand you a transferable cutoff. The size of these blow-ups is telling: a Gaussian-tail calculation maps the macro FPR shift (8.1% $\rightarrow$ 15.5%) to only $\approx$ 0.4 standard deviations of calibration drift, and the pooled shift (4.3% $\rightarrow$ 17.0%) to $\approx$ 0.8 SD — sub-sigma drifts that no realistic calibration protocol excludes, because a fixed cutoff's false-alarm rate is *exponentially* sensitive to drift at the operating quantile.

9. **Prompt-only scope.** We classify the *incoming request* (binary `safe`/`unsafe`) via the single-token $z_{\text{unsafe}} - z_{\text{safe}}$ head. We do not evaluate response classification, multi-turn context, tool-call arguments, or streamed content. Guards that read the assistant's *output* or a full dialogue are a different task and are out of scope; nothing here should be read as a claim about them.

Act I (SFT specialization) — what it does not establish. The headline aggregate transfer change of -0.0589 is *not* a clean "fine-tuning barely hurts transfer." It pools opposing per-checkpoint effects (Qwen2.5-1.5B -0.0389 $[-0.0829, +0.0062]$; SmolLM2-1.7B $+0.0400$ $[+0.0003, +0.0776]$; SmolLM3-3B -0.0869 $[-0.1114, -0.0613]$; Qwen3-4B -0.1499 $[-0.1963, -0.1050]$); the panel average is small precisely because it averages a gain against three losses. The specialization plane (Figure 3) shows 15 of 20 (checkpoint, seed) points in the specialize quadrant and 5 in uniform gain — a *tendency on this panel*, not a universal direction, and one seed-family (SmolLM2) runs the other way.

Act II (composition) — what it does not establish. Composition (Equation (5)) recovers transfer relative to SFT ($+0.076$ $[+0.058, +0.093]$) at a small represented cost (-0.019 $[-0.031, -0.010]$), landing above the base on transfer (0.883 vs. 0.866). But: **(a)** it is *not* Pareto dominance — versus base it is heterogeneous, helping the weaker bases (SmolLM2 $+0.067$, Qwen2.5 $+0.036$ vs. base transfer) while *hurting* the strongest, Qwen3-4B (-0.030); **(b)** the mechanism is unproven (the ensemble/diversity account is a motivation, not a proof); **(c)** one control is *run* and one is still missing — we have run the equal-inference-cost SFT+SFT baseline (Table 8: base+SFT beats SFT+SFT on all four checkpoints, attributing the recovery to "keeping the base" rather than to "ensembling anything"), but not a true weight-space WiSE-FT rescoring [46], so we cannot say which family recovers more transfer here; and **(d)** the logit-average and convex-weight variants (transfer 0.891 for logit-average) were *visible during development* and are reported only as non-promotable ablations. Whether composition also recovers transfer for guards tuned with *other* objectives is left open.

Act III, mortgage — a measuring stick, not a legal finding. The mortgage benchmark's 994 dual labels (G=general-safety, D=mortgage-policy) are assigned by an *LLM judge against written policy cards*, with self-consistency checks but *no* subject-matter-expert (SME) adjudication and *no* human inter-annotator agreement (no Fleiss-κ). It therefore *surfaces* guard behavior on the load-bearing G0/D1 stratum (reads safe, is a violation); it *certifies nothing* about any real lender, applicant, or model, and licenses no fair-lending or disparate-impact conclusion. Four further limits bound it: **(i)** the **G1/D0 quadrant is empty** (0 rows), so "looks unsafe yet is policy-compliant" — the over-refusal-on-compliant-requests failure mode — is *unmeasurable* here; **(ii)** several strata are *small* (G1/D1 = 42 rows; private_test = 95 rows; 39 protected minimal pairs / 78 rows), so per-stratum estimates are noisy; **(iii)** the zero-shot bases rank policy violations only *moderately* (AP·D 0.67–0.85; Table 12), and because asking for discrimination or fraud already reads as unsafe to a general model, G and D are only *partially* orthogonal — much of the subtle G0/D1 stratum stays unresolved by ranking; **(iv)** the fixed-threshold operating point is deliberately *not* tabulated because it is knife-edge for these clustered-score guards (its G0/D1 catch count swung by more than 50 rows across library versions) — itself a finding about unreliable threshold transfer, not a number to deploy. The protected-pair gap Δ_{context} is a fairness *signal* (Qwen3-4B invariant at 0.000; Qwen2.5-1.5B 0.183; max observed 0.51), not a fairness *verdict*.

Act III, ExpGuard — external, single-label, base-only, never pooled. The finance/health/law replication uses *external, expert-annotated* labels (`prompt_label`) over 2,275 rows — a strictly stronger labeling tier than the mortgage LLM-judge. It tests whether the specialization/transfer pattern *recurs* across regulated verticals, but it is **single-label** (not the mortgage dual $G \times D$ construct), so it is a breadth/transfer probe, not a compliance construct. The four-checkpoint *base* result is complete (Table 13) and reproducible from committed per-row scores; the tuned (base-vs-SFT) comparison on ExpGuard is future work. Because ExpGuard's labels come from a different source and tier, its numbers are *never* averaged, ranked, or otherwise pooled with the mortgage LLM-judge numbers or with the retrospective panel.

The one-line version

We measured paired base-to-tuned changes on a fixed two-lineage panel, on benchmarks we had seen, at balanced prevalence, using ranking metrics — plus one depth domain with LLM-judge labels and one external breadth domain with expert labels. That supports "on this panel, tuning specializes and composition recovers some transfer." It does *not* support any causal, universal, deployment, or fair-lending claim.

E.2 The unified evidence ledger

The report carries evidence at *different tiers*, and the honesty of the synthesis depends on never letting a weaker tier borrow credibility from a stronger one. Two dimensions matter. The first is the **evidence flavor**: is the number *retrospective / estimation-only* (measured after the fact on inspected data, conditional on a fixed panel) or does it come from an *external, independently annotated* source? The second is the **label tier**, in decreasing strength: *SME-adjudicated with reported inter-annotator agreement* (the gold standard — we have *none* of this yet); *external expert-annotated* (ExpGuard); and *LLM-judge, policy-card-consistent* (the mortgage benchmark — self-consistent against written cards, but no human). Table 22 records, for each body of evidence, its flavor, its label tier, what it establishes, and what would upgrade it. The governing rule is stated once and applied everywhere: **the three columns of flavor are never pooled**. Retrospective panel numbers (Acts I–II), the LLM-judge mortgage numbers (Act III depth), and the external expert-annotated ExpGuard numbers (Act III breadth) are reported side by side but never averaged into a single headline.

Why "never pool" is not just bookkeeping

If you average an LLM-judge score (which can inherit the judge model's blind spots) with an expert-annotated score (which cannot), the blended number is neither. Worse, it launders the weaker label's uncertainty into the stronger one and produces a headline no single method would support. Keeping the flavors in separate columns means a reader can always ask "how was this labeled?" and get a straight answer — and it means the mortgage benchmark's honest status (a measuring stick, not a certification) is never quietly upgraded by proximity to the expert-annotated set.

E.3 The validation roadmap

Each limitation above has a matching upgrade. The roadmap is ordered roughly by how much it would strengthen the central claims, and every item names the specific weakness it closes.

1. **Lock a genuinely uninspected prospective cohort** (closes "not confirmatory," Acts I–II). Pre-register the estimands and decision rules, then evaluate on data neither the researcher nor the manifest builder has seen. This is the single upgrade that would move Acts I and II from "retrospective characterization of this panel" to a confirmatory finding; the adaptation study

Table 22: The unified evidence ledger. Each row records one body of evidence, its flavor and label tier, what it establishes, and the concrete upgrade that would strengthen it. The three flavors are never pooled; no row is promoted to a causal, universal, or fair-lending claim.

Body of evidence	Flavor & label tier	Establishes (conditional on panel/data)	Principal limits → upgrade
Act I: SFT specialization (Tables 1 and 3, Figure 3)	Retrospective, estimation-only; dataset-held-out; benchmarks inspected in dev	On this panel, SFT lifts represented-source AP (+0.3234) but not transfer (−0.0589); 15/20 seeds specialize	Two lineages; single recipe; balanced prevalence; Δ coupled to base; v2 decontamination audit pending → prospective uninspected cohort + overlap audit
Act II: composition (Tables 6, 7 and 9, Equation (5))	Retrospective, estimation-only; same panel	Vs. SFT, recovers transfer (+0.076) at small represented cost (−0.019); above base on transfer	Not Pareto (hurts Qwen3-4B); ablations dev-visible; SFT+SFT control run (Table 8, base+SFT wins on all four); WiSE-FT rescoring still open → add weight-space control
Act III depth: mortgage $G \times D$ (Tables 11 and 12, Figure 13)	LLM-judge, policy-card-consistent; *not* SME; 994 rows	A dual-labeled, HMDA-grounded stick for the G0/D1 stratum + a protected-pair fairness signal (Δ_{context})	No human agreement; empty G1/D0; small strata; knife-edge threshold → SME adjudication + Fleiss-κ; populate G1/D0
Act III breadth: ExpGuard (Table 13)	External, expert-annotated; single-label; base-only (complete); 2,275 rows	Whether the specialization/-transfer pattern recurs across finance/health/law with expert labels	Single-label (not dual $G \times D$); base-only → add the tuned (base-vs-SFT) comparison; dual-label expert construct

(Section 4) already applies this discipline to the starting-type question, and the remaining step is to extend it to Acts I–II's core specialization estimands on genuinely uninspected data.

2. **Complete the v2 decontamination audit** (closes the pending-audit caveat). Run the formal n-gram / near-duplicate overlap check of the 1,200-row manifest against the v2 transfer suite and report the residual overlap; any leakage found would revise the transfer estimates *downward* (i.e. specialization would look sharper).

3. **Add the remaining composition control** (closes Act II's mechanism gap). The equal-inference-cost **SFT+SFT** baseline is now run (Table 8: base+SFT beats two-adapter ensembling on every checkpoint, so the recovery is the base's doing); the open item is a true weight-space **WiSE-FT** rescoring [46] to compare output-space against weight-space interpolation on identical rows.

4. **SME-adjudicate a stratified mortgage subset and report Fleiss-κ** (closes the mortgage label-tier gap). Have qualified subject-matter experts independently re-label a stratified subset (weighted toward the G0/D1 and protected-pair rows), report inter-annotator agreement, and reconcile against the LLM-judge labels; this is what would let any mortgage number graduate from "measuring stick" toward an audit-grade instrument.

5. **Populate the empty G1/D0 quadrant** (closes an unmeasurable failure mode). Construct grounded "looks unsafe yet policy-compliant" requests so over-refusal on legitimate mortgage traffic becomes measurable rather than absent by construction.

6. **Evaluate at production prevalence with a cost-weighted operating point** (closes

"balanced prevalence overstates precision"). Re-score at realistic (heavily benign) base rates and report precision and a cost-weighted threshold, so the deployed-precision story is not read off a balanced pool.

7. **Broaden the panel beyond two lineages** (closes the lineage/scale confound). Add checkpoints from additional model families and a wider size range so that "stronger bases specialize more" can be tested rather than inferred from four coupled points.

8. **Extend ExpGuard to the tuned comparison and build dual-labeled finance/health constructs with expert sign-off** (closes the breadth gap). The four-checkpoint *base* ExpGuard eval is complete (Table 13); add the tuned (base-vs-SFT) comparison, and extend the mortgage-style dual-label $G \times D$ construct into finance and health with expert adjudication — one domain built in depth plus verticals built for breadth.

9. **Score gated / commercial guards and extend beyond prompt-only** (closes the non-commercial-data and scope limits). Once licenses are accepted, evaluate gated open guards (e.g. Llama Guard 3, WildGuard) on the same rows and scorer; and extend the task beyond input-prompt classification to response and multi-turn moderation. Note that several training sources are non-commercial / gated, which constrains redistribution — raw third-party rows are referenced by pinned identifier + revision + content hash, never redistributed.

The disciplined next step

The honest upgrade path is not a more confident headline; it is a *prospectively locked, decontaminated, appropriately controlled* re-run on genuinely uninspected data, with SME-adjudicated labels where compliance is at stake. Until then, every claim in this report stands exactly as written: a reproducible, estimation-only characterization of one fixed panel, plus one LLM-judge depth domain and one external expert-annotated breadth domain — reported honestly, and never pooled.

References

[1] Akindoyin Akinrele and Shreyank N. Gowda. Prompt injection detection is regime-dependent: A deployment-aware evaluation with interpretable structural signals. arXiv:2605.26999, 2026.

[2] Elie Bakouch, Loubna Ben Allal, Anton Lozhkov, Nouamane Tazi, Lewis Tunstall, Carlos Miguel Patiño, Edward Beeching, Aymeric Roucher, Aksel Joonas Reedi, Quentin Gallouédec, Kashif Rasul, Nathan Habib, Clémentine Fourrier, Hynek Kydlíček, Guilherme Penedo, Hugo Larcher, Mathieu Morlon, Vaibhav Srivastav, Joshua Lochner, Xuan-Son Nguyen, Colin Raffel, Leandro von Werra, and Thomas Wolf. Smollm3: smol, multilingual, long-context reasoner. Hugging Face blog + model card (HuggingFaceTB/SmolLM3-3B), https://huggingface.co/blog/smollm3, 2025.

[3] Elias Bassani and Ignacio Sanchez. GuardBench: A large-scale benchmark for guardrail models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 18393–18409, 2024.

[4] Elias Bassani and Ignacio Sanchez. On guardrail models' robustness to mutations and adversarial attacks. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 16995–17006, Suzhou, China, 2025. Association for Computational Linguistics. doi: 10.18653/v1/2025.findings-emnlp.922. URL `https://aclanthology.org/2025.findings-emnlp.922/`.

[5] Donald E. Bowen III, S. McKay Price, Luke C.D. Stein, and Ke Yang. Measuring and mitigating racial bias in large language model mortgage underwriting, 2024. Working paper.

[6] Patrick Chao, Edoardo Debenedetti, Alexander Robey, Maksym Andriushchenko, Francesco Croce, Vikash Sehwag, Edgar Dobriban, Nicolas Flammarion, George J. Pappas, Florian Tramèr, Hamed Hassani, and Eric Wong. Jailbreakbench: An open robustness benchmark for jailbreaking large language models. In *NeurIPS 2024 (Datasets & Benchmarks)*, 2024.

[7] Minseok Choi, Dongjin Kim, Seungbin Yang, Subin Kim, Youngjun Kwak, Juyoung Oh, Jaegul Choo, and Jungmin Son. ExpGuard: LLM content moderation in specialized domains, 2026. ICLR 2026; arXiv:2603.02588.

[8] Justin Cui, Wei-Lin Chiang, Ion Stoica, and Cho-Jui Hsieh. Or-bench: An over-refusal benchmark for large language models. In *ICML 2025 (PMLR 267)*, pages 11515–11542, 2025.

[9] Yihe Deng, Yu Yang, Junkai Zhang, Wei Wang, and Bo Li. Duoguard: A two-player rl-driven framework for multilingual llm guardrails. *arXiv preprint*, 2025.

[10] Zhihao Ding, Jinming Li, Ze Lu, and Jieming Shi. FlexGuard: Continuous risk scoring for strictness-adaptive LLM content moderation. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 5825–5851, San Diego, California, United States, 2026. Association for Computational Linguistics. doi: 10.18653/v1/2026.acl-long.263. URL `https://aclanthology.org/2026.acl-long.263/`.

[11] Huaixia Dou, Jie Zhu, Minghao Wu, Shuo Jiang, Junhui Li, Lifan Guo, Feng Chen, and Chi Zhang. FinGuard: Detecting financial regulatory non-compliance in LLM interactions. arXiv:2605.29427, 2026.

[12] Hayder Elesedy, Pedro M. Esperanca, Silviu Vlad Oprea, and Mete Ozay. LoRA-guard: Parameter-efficient guardrail adaptation for content moderation of large language models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 11746–11765, Miami, Florida, USA, 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.emnlp-main.656. URL `https://aclanthology.org/2024.emnlp-main.656/`.

[13] Federal Financial Institutions Examination Council (FFIEC) and Consumer Financial Protection Bureau (CFPB). Hmda snapshot national loan-level dataset (2022). `https://ffiec.cfpb.gov/data-publication/snapshot-national-loan-level-dataset/`, 2023.

[14] Igor Fedorov, Kate Plawiak, Lemeng Wu, Tarek Elgamal, Naveen Suda, Eric Smith, Hongyuan Zhan, Jianfeng Chi, Yuriy Hulovatyy, Kimish Patel, Zechun Liu, Changsheng Zhao, Yangyang Shi, Tijmen Blankevoort, Mahesh Pasupuleti, Bilge Soran, Zacharie Delpierre Coudert, Rachad Alao, Raghuraman Krishnamoorthi, and Vikas Chandra. Llama guard 3-1b-int4: Compact and efficient safeguard for human-ai conversations. *arXiv preprint*, 2024.

[15] Shaona Ghosh, Prasoon Varshney, Erick Galinkin, and Christopher Parisien. Aegis: Online adaptive ai content safety moderation with ensemble of llm experts. *arXiv preprint*, 2024.

[16] Ryle Goehausen and Marcus Sousa. Gate AI: LLM security benchmark evaluation methodology and results. arXiv:2606.02959, 2026.

[17] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *ICML 2017 (PMLR 70)*, pages 1321–1330, 2017.

[18] William Hackett, Lewis Birch, Stefan Trawicki, Neeraj Suri, and Peter Garraghan. Bypassing llm guardrails: An empirical analysis of evasion attacks against prompt injection and jailbreak detection systems. In *Proceedings of the First Workshop on LLM Security (LLMSEC 2025)*, pages 101–114, 2025.

[19] Seungju Han, Kavel Rao, Allyson Ettinger, Liwei Jiang, Bill Yuchen Lin, Nathan Lambert, Yejin Choi, and Nouha Dziri. Wildguard: Open one-stop moderation tools for safety risks, jailbreaks, and refusals of llms. In *NeurIPS 2024 (Datasets & Benchmarks Track)*, 2024.

[20] Lei Hsiung, Tianyu Pang, Yung-Chen Tang, Linyue Song, Tsung-Yi Ho, Pin-Yu Chen, and Yaoqing Yang. Why llm safety guardrails collapse after fine-tuning: A similarity analysis between alignment and fine-tuning datasets. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 16601–16619, 2026. doi: 10.18653/v1/2026.acl-long.756. Earlier workshop version at ICML 2025 DIG-BUGS.

[21] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. In *ICLR 2022*, 2021.

[22] Hakan Inan, Kartikeya Upasani, Jianfeng Chi, Rashi Rungta, Krithika Iyer, Yuning Mao, Michael Tontchev, Qing Hu, Brian Fuller, Davide Testuggine, and Madian Khabsa. Llama guard: Llm-based input-output safeguard for human-ai conversations. *arXiv preprint*, 2023.

[23] Jiaming Ji, Mickel Liu, Juntao Dai, Xuehai Pan, Chi Zhang, Ce Bian, Ruiyang Sun, Boyuan Chen, Yizhou Wang, and Yaodong Yang. Beavertails: Towards improved safety alignment of llm via a human-preference dataset. In *NeurIPS 2023 (Datasets & Benchmarks)*, 2023.

[24] Liwei Jiang, Kavel Rao, Seungju Han, Allyson Ettinger, Faeze Brahman, Sachin Kumar, Niloofar Mireshghallah, Ximing Lu, Maarten Sap, Yejin Choi, and Nouha Dziri. Wildteaming at scale: From in-the-wild jailbreaks to (adversarially) safer language models. In *NeurIPS 2024*, 2024.

[25] Mintong Kang and Bo Li. R^2-guard: Robust reasoning enabled llm guardrail via knowledge-enhanced logical reasoning. In *ICLR 2025*, 2025.

[26] Ananya Kumar, Tengyu Ma, Percy Liang, and Aditi Raghunathan. Calibrated ensembles can mitigate accuracy tradeoffs under distribution shift. In *Uncertainty in Artificial Intelligence (UAI), PMLR 180*, pages 1041–1051, 2022.

[27] Li Li, Chenxiao Yu, Zhiyu Ni, Hao Li, Charith Peris, Chaowei Xiao, and Yue Zhao. Defenses against prompt attacks learn surface heuristics. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 10970–10987, San Diego, California, United States, 2026. Association for Computational Linguistics. doi: 10.18653/v1/2026.acl-long.502. URL `https://aclanthology.org/2026.acl-long.502/`.

[28] Zi Lin, Zihan Wang, Yongqi Tong, Yangkun Wang, Yuxin Guo, Yujia Wang, and Jingbo Shang. Toxicchat: Unveiling hidden challenges of toxicity detection in real-world user-ai conversation. In *Findings of EMNLP 2023*, pages 4694–4702, 2023.

[29] Hongfu Liu, Hengguan Huang, Xiangming Gu, Hao Wang, and Ye Wang. On calibration of LLM-based guard models for reliable content moderation. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2410.10414.

[30] Minqian Liu, Ioana Baldini, David Rabinowitz, David S. Rosenberg, Sebastian Gehrmann, and Mark Dredze. Domain generalizable AI guardrails with augmented policy training. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 16452–16469, San Diego, California, United States, 2026. Association for Computational Linguistics. doi: 10.18653/v1/2026.acl-long.748. URL `https://aclanthology.org/2026.acl-long.748/`.

[31] Vasudev Majhi, Dhruv Gupta, Advait Singh, Matthew Barker, and Dhruv Kumar. Do you really need a gpu to guard your llm? cpu-class classifiers and multi-stage pipelines for safety enforcement at scale. *arXiv preprint*, 2025.

[32] Mantas Mazeika, Long Phan, Xuwang Yin, Andy Zou, Zifan Wang, Norman Mu, Elham Sakhaee, Nathaniel Li, Steven Basart, Bo Li, David Forsyth, and Dan Hendrycks. Harmbench: A standardized evaluation framework for automated red teaming and robust refusal. In *ICML 2024 (PMLR 235)*, pages 35181–35224, 2024.

[33] Meta AI. Llama guard 2 model card. GitHub model card, 2024. PurpleLlama repository, Llama-Guard2 model card.

[34] Meta AI. Llama guard 3-1b model card. Hugging Face model card, 2024. https://huggingface.co/meta-llama/Llama-Guard-3-1B.

[35] Meta AI. Llama guard 3-8b model card. GitHub model card, 2024. PurpleLlama repository, Llama-Guard3 8B model card.

[36] Meta AI. Prompt guard 86m model card. Hugging Face model card, 2024. https://huggingface.co/meta-llama/Prompt-Guard-86M.

[37] Meta AI. Llama guard 4: A 12b multimodal safety classifier. Meta / Hugging Face model card, 2025. https://huggingface.co/meta-llama/Llama-Guard-4-12B.

[38] Meta AI. Llama prompt guard 2-22m model card. Hugging Face model card, 2025. https://huggingface.co/meta-llama/Llama-Prompt-Guard-2-22M.

[39] Meta AI. Llama prompt guard 2-86m model card. Hugging Face model card, 2025. https://huggingface.co/meta-llama/Llama-Prompt-Guard-2-86M.

[40] Inkit Padhi, Manish Nagireddy, Giandomenico Cornacchia, Subhajit Chaudhury, Tejaswini Pedapati, Pierre Dognin, Keerthiram Murugesan, Erik Miehling, Martín Santillán Cooper, Kieran Fraser, Giulio Zizzo, Muhammad Zaid Hameed, Mark Purcell, Michael Desmond, Qian Pan, Zahra Ashktorab, Inge Vejsbjerg, Elizabeth M. Daly, Michael Hind, Werner Geyer, Ambrish Rawat, Kush R. Varshney, and Prasanna Sattigeri. Granite guardian: Comprehensive llm safeguarding. In *NAACL 2025 (Industry Track)*, pages 607–615, 2025.

[41] Qwen Team. Qwen3guard technical report, 2025. https://github.com/QwenLM/Qwen3Guard.

[42] Reza Rahimi. The benchmark chooses the winner: Measuring fine-tuning specialization, not general improvement, in small safety guards. Companion manuscript and reproducibility artifacts, 2026. Retrospective clean-v2 study.

[43] Paul Röttger, Hannah Rose Kirk, Bertie Vidgen, Giuseppe Attanasio, Federico Bianchi, and Dirk Hovy. Xstest: A test suite for identifying exaggerated safety behaviours in large language models. In *NAACL 2024 (Long Papers)*, pages 5377–5400, 2024.

[44] Matthew Toles, Yunan Lu, Manav Munjal, Bojun Liu, Yuanhao Deng, Stephanie Selig, Derek Rindner, Cheng Li, and Zhou Yu. MortarBench: Evaluating mortgage loan origination agents. arXiv:2606.19416, 2026.

[45] Mitchell Wortsman, Gabriel Ilharco, Samir Ya Gadre, Rebecca Roelofs, Raphael Gontijo-Lopes, Ari S. Morcos, Hongseok Namkoong, Ali Farhadi, Yair Carmon, Simon Kornblith, and Ludwig Schmidt. Model soups: Averaging weights of multiple fine-tuned models improves accuracy without increasing inference time. In *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pages 23965–23998, 2022. URL `https://proceedings.mlr.press/v162/wortsman22a.html`.

[46] Mitchell Wortsman, Gabriel Ilharco, Jong Wook Kim, Mike Li, Simon Kornblith, Rebecca Roelofs, Raphael Gontijo-Lopes, Hannaneh Hajishirzi, Ali Farhadi, Hongseok Namkoong, and Ludwig Schmidt. Robust fine-tuning of zero-shot models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022. URL `https://arxiv.org/abs/2109.01903`.

[47] Wenjun Zeng, Yuchi Liu, Ryan Mullins, Ludovic Peran, Joe Fernandez, Hamza Harkous, Karthik Narasimhan, Drew Proud, Piyush Kumar, Bhaktipriya Radharapu, Olivia Sturman, and Oscar Wahltinez. Shieldgemma: Generative ai content moderation based on gemma. *arXiv preprint*, 2024.